wick プログラミング言語

lantern エンジンのための
小さな型付きスクリプト言語

lantern エンジン開発者

バージョン 0.3 • 2026
日本語版

wick プログラミング言語、バージョン 0.3、日本語版。

wick と lantern エンジンは、zlib ライセンスで公開するオープンソースです。字句解析器、一回走査の型付きコンパイラ、バイトコード、スタック仮想マシンからなる言語実装は、依存ライブラリのない約二千行の C++ で、エンジンとともに `github.com/alikatgh/lantern` にあります。

本書の日本語版は、LaTeX と XeLaTeX を使い、日本語の明朝体と Latin Modern で組版しています。すべてのコード例は Wick 0.3 向けです。診断の例は、意図的にエラーを示しています。

Copyright © 2026 The lantern engine authors. 本書は、エンジンと同じ zlib の条件で配布します。出典や著作者を偽らない限り、このテキストを使用、複製、変更できます。

目次

1 はじめに **1**
1.1 wick が生まれた理由 1
1.2 Lua から学んだこと 2
1.3 設計の原則 3
1.4 本書の読み方 4

2 wick チュートリアル **7**
2.1 導入と実行 7
2.2 ゲームは一つのフォルダ 7
2.3 変数と最初の動き 8
2.4 関数 9
2.5 オプショナル値と得点の保存 10
2.6 リストでランプを持つ 11
2.7 一ラウンドを組み立てる 12
2.8 開発のループ 13

3 言語 **15**
3.1 字句構造 15
3.2 型 16
3.3 変数とスコープ 18
3.4 式と優先順位 19
3.5 文 21
3.6 制御フロー 21
3.7 関数 22
3.8 代入の互換性 24

4 オプショナル値 **25**
4.1 Lua で起きた問題 25
4.2 バグを書けなくする型 26
4.3 `??` で中の値を取り出す 26
4.4 `if x != nil` で型を絞り込む 27
4.5 オプショナル値を返す操作 28

4.6 型システムの中のオプショナル値 29
4.7 実際の書き方 29

5 コレクション **31**
5.1 リスト 31
5.2 マップ 32
5.3 レコードとレコードのリスト 33
5.4 反復のパターン 34
5.5 入れ子のコンテナを使わない入れ子データ 35

6 エンジンインターフェース **37**
6.1 フレームの契約 37
6.2 フレームと画面 38
6.3 3D シーン 38
6.4 メッシュ 39
6.5 2D 40
6.6 音声、入力、保存 41
6.7 決定性 41
6.8 検証を日常にする 42

7 内部の仕組み **45**
7.1 一回走査の型付きコンパイル 45
7.2 特化した命令 46
7.3 スタック VM 47
7.4 フレーム境界でのガベージコレクション 47
7.5 wick の組み込み 48

8 設計とトレードオフ **51**
8.1 同じゲームを二つの言語で 51
8.2 書かなくなるもの 52
8.3 変わらないもの 53
8.4 v0.3 の制約 53
8.5 ロードマップの考え方 54

9 ビット、バイト、バス **57**
9.1 意図するビットを書く 57
9.2 ゲートとマスク 57
9.3 レジスタには幅がある 58
9.4 ペアとメモリ 59
9.5 状態を読みやすくする 59

9.6 作業台と次のゲーム 59

A 文法 **61**

B コンパイルエラー一覧 **65**

C 早見表 **69**
C.1 ビットとレジスタ幅（0.3） 71

第 1 章

はじめに

ランタンの芯（wick）は、炎を支える部分です。 wick は lantern エンジン専用のスクリプト言語です。Lua の小ささと使い心地を受け継ぎながら、つまずきやすい部分を lint ルールで補うのではなく、言語そのものの設計で取り除いています。本章では、ゲームエンジンが独自の言語を持つに至った理由、その言語が解決しようとする問題、そして本書の構成を説明します。

1.1 wick が生まれた理由

wick を作ったのは、世の中にもう一つ言語が必要だったからではありません。Lua で実際にゲームを出荷し、困ったことを記録していった結果、どの項目も同じ構図だと気づいたからです。言語が許してしまう誤りがあり、それが見つかるのはプレイヤーが遭遇したときだけでした。

Lua はすばらしい言語です。小さく、組み込みやすく、高速で、C との連携もとても簡単です。この十年、「ゲームのスクリプトには何を使うべきか」という問いへの定番の答えでした。その長所に異を唱えるつもりはありません。ただ、ゲームエンジンのスクリプト言語には固有の役割があり、固有の失敗のしかたがあります。汎用性と誕生当時の事情を踏まえて選ばれた Lua の既定の振る舞いが、その用途にはまさに不向きだったのです。

何度も立ち戻る失敗があります。第 4 章の中心となる例です。私たちのデモゲーム Lantern Night は、プレイヤーの最高得点をファイルに保存します。Lua 版では、読み込みを次のように書いていました。

```
local best = tonumber(lt.load_save("lanternnight_best") or "0")
```

正しく見えます。レビューも通り、出荷もされました。それでも保存が書き込み途中で中断されると、最初のフレームでクラッシュしました。途中で中断された書き込みは 0 バイトのファイルを残します。`lt.load_save` は `nil` ではなく空文字列 `""` を返します。Lua では空文字列が真として

扱われるため、`or "0"` の代替値は使われません。そして `tonumber("")` は `nil` になり、それ以降の算術式をすべて壊してしまいます。空文字列を真とすること、エラーを `nil` で表すこと、`nil` が黙って伝播すること、静的検査がないこと。この四つの設計判断が重なり、一度のディスク書き込みの中断が、コードを読むだけでは見抜けない最初のフレームのクラッシュに変わりました。

wick では、このバグを書けません。保存専用の例外処理を追加したからではありません。`nil` かもしれない値を使うとき、先に `nil` の場合を扱わなければ、型システムがコンパイルを許さないからです。wick でこの行を書くなら、コンパイラが受け付ける形は次のとおりです。

```
let best = num(lt.load_save("lanternnight_wick_best") ?? "") ?? 0
```

この一行に、言語の基本思想がすべて詰まっています。以下の設計判断は、どれもこの思想に沿っています。

1.2 Lua から学んだこと

Lua で困った点と、そこから生まれた wick の判断をまとめます。どれも lint ルールやスタイルガイドではありません。コンパイラまたは実行時の仕組みが強制するため、締め切りに追われても忘れようがありません。

Lua で困った点	wick の答え
実行時の `nil` エラー	静的な型と `T?` のオプショナル値。未確認の使用はコンパイルできない。
タイプミスが暗黙のグローバル変数を作る	宣言した変数のみ。未宣言の名前への代入はコンパイルエラー。
添字は 1 から、`continue` がない	添字は 0 からの `[]`、`continue`、`len(x)`。
`""` と `0` が真になる	条件は `bool` に限定。真偽の暗黙変換はない。
フレーム途中の GC 停止	回収はフレームの*間だけ*で実行。
スタック添字による C バインディング	エンジン呼び出しの型をコンパイル時に検査。
機種によって `math.random` が異なる	`rand()` は固定の xorshift。どこでも決定的に動作。
iOS では JIT が禁止	JIT を使わず、静的な型に応じたバイトコードを生成。

ここには共通するパターンがあります。各行の Lua の振る舞いは、抽象的には合理的でも、ゲームループでは危険です。真偽の暗黙変換は便利ですが、得点が 0 のときに気づかないまま別の分岐へ進むと困ります。暗黙のグローバル変数は短く書けますが、タイプミスが `nil` になり、三つ先の関数で表面化することがあります。好きなときに動くガベージコレクタはバッチ処理なら問題なくても、アクションゲームではフレーム落ちの原因です。wick の答えはいつも同じです。Lua が実行時に動的に行うことを、コンパイル時に静的に行う。あるいは GC なら、ホストが選んだ時点で行います。

1.3 設計の原則

言語全体を貫く原則は四つあります。後の章で、ある機能がなぜそのように動くのかを説明するとき、答えはほぼ必ずこのいずれかです。

小さいこと。 wick は意図的に Lua 1.0 のような小ささにしています。字句解析器、コンパイラ、仮想マシン、ガベージコレクタ、組み込み関数を含む実装全体が、依存ライブラリのない約二千行の C++ です。v0.3 にはクロージャ、クラス、モジュール、メタテーブル、可変長引数、文字列メソッドがありません。省いた機能はすべて、第 8 章に回避策とロードマップ上の位置づけを記載しています。小ささは、偶然そうなったことへの言い訳ではなく、守ろうとしている特徴です。午後だけで読み切れる言語なら、午前 3 時にも振る舞いを予測できます。

型があること。 どの式にも静的な型があり、コンパイラがその式のコードを生成する時点で型は分かっています。推論の材料がある限り型を推論するので、書き方は軽いままです。`let x: num = 10` と書かずに `let x = 10` と書けます。それでも検査は全体に及びます。型システムの中心はオプショナル値 `T?` です。「値が存在しないかもしれない」という事実をコンパイラが追跡し、プログラマがその可能性を処理する必要があります。

決定的であること。 同じプログラムに同じ入力を与えれば、どの機種でも同じフレームを生成します。`rand()` は固定の xorshift 生成器で、時刻によるシード設定はありません。実行ごとに変化をつけたい場合は、自分で選び記録できる数値を `srand()` に渡します。`LANTERN_FIXED_DT` を使うとフレーム時刻の刻みも固定され、ゲームプレイ全体をバイト単位

で同じように再現できます。これがスクリーンショットを使った CI を可能にし、第 6 章ではその上にテストの習慣を築きます。

フレームを意識すること。 wick は毎秒 60 フレームのループの中で動くことを前提にしています。ガベージコレクタはホストが要求したときだけ動きます。lantern が要求するのは、画像を表示した後、各フレームに一度だけです。停止が画面に現れないタイミングを選んでいます。エンジン API は、ホストが毎フレーム呼ぶ二つの関数 `update(dt)` と `draw()` を中心に構成されています。ゲームにたまたま埋め込まれた汎用ツールではなく、字句解析器の段階からゲームループに合わせた言語です。

1.4 本書の読み方

最初は先頭から順に読み、その後はリファレンスとして使うことを想定しています。

- **第 2 章**はチュートリアルです。エンジンを導入し、最初のプログラムを動かし、小さなランプゲームを段階的に完成させます。厳密なリファレンスの章に進む前に、動くコードを手元に用意できます。
- **第 3 章**は言語リファレンスです。字句構造、型、スコープ、式、優先順位、文、制御フロー、関数を扱います。
- **第 4 章**は wick の核であるオプショナル値を扱います。本章で紹介した 0 バイトの保存ファイルの実例も交えて説明します。
- **第 5 章**はコレクションであるリストとマップ、そして構造体の代わりに使う並行リストの書き方を扱います。
- **第 6 章**は `lt.*` のエンジンインターフェースを説明します。フレームの契約、型付き API の表、決定性、検証を扱います。
- **第 7 章**は実装を開いて見ます。一回走査の型付きコンパイル、スタック VM、フレーム境界でのガベージコレクション、自分の C++ への wick の組み込み方を説明します。
- **第 8 章**は設計の根拠です。同じ実際のゲームで wick と Lua を比べ、率直なトレードオフとロードマップの考え方を述べます。

- **付録**には EBNF による完全な文法、コンパイルエラーの全一覧、組み込み関数とエンジン API の早見表があります。

本書のコードは、別の言語だと明記されない限り、この書体で示した wick のコードです。どのコード例も実際の v0.3 の wick で、コンパイルできるものです。コンパイルエラーを示すため意図的に間違えた例には、対応するエラーメッセージを添えています。Lua の例は wick の例に対応するもので、Lua と明記しています。

率直さについて一つ。本書では、今の wick より Lua の方が優れている点を何度か述べます。動的なテーブルは入れ子のデータをテーブルとして保持できます。複数の戻り値は実際に不足しています。Lua のエコシステムは巨大で、こちらは一つのエンジンです。対象を褒めるだけの言語の本は、あまり役に立ちません。wick が存在する価値は、特定の、しかも高くつく種類のバグを取り除くことにあります。何を取り除き、何を取り除かないのかは、正確に述べるべきです。

第 2 章

wick チュートリアル

本章では、空のフォルダから遊べるものになるまで、小さくても完全なゲームを作ります。言語機能は必要になった時点で紹介し、ゲームに必要な範囲を説明します。厳密な規則は第 3 章で扱います。最後には、変数、関数、オプショナル値、リスト、ループ、そしてエンジンが毎フレーム呼ぶ二つの関数を書き終えています。さらに、wick での開発を快適にするホットリロードと画面上のエラー表示も使います。

2.1 導入と実行

wick は lantern エンジンに含まれています。別途インストールするものはありません。エンジンをビルドできれば、wick を実行できます。

```
git clone https://github.com/alikatgh/lantern && cd lantern
brew install sdl2 lua cmake pkg-config   # macOS; Linux: same
    packages
cmake -B build && cmake --build build -j8

./build/lantern games/wicklab            # the first wick program
./build/lantern games/showcase_wick      # Lantern Night, in wick
```

最初のコマンドはエンジンをビルドします。最後の二つは、リポジトリにあるゲームを実行します。`wicklab` は言語を短く紹介するデモで、`showcase_wick` は本書で繰り返し登場する Lantern Night の wick 版です。

2.2 ゲームは一つのフォルダ

wick のゲームは、`main.wick` というファイルを含むディレクトリです。エンジンにフォルダを指定すると、そのファイルがあれば実行し、なけ

れば main.lua を使います。新しいゲームを始めるには、次のようにします。

```
mkdir games/lamp
$EDITOR games/lamp/main.wick
./build/lantern games/lamp
```

画面は 400×240 ピクセルです。エンジンは毎秒 60 フレームで動作し、毎フレーム、名前で二つの関数を探します。

```
fn update(dt: num) {   // dt = seconds since last frame, capped
    at 0.1
}

fn draw() {            // 3D first, then 2D composites on top
  lt.clear(0.1, 0.1, 0.2)
  lt.print("HELLO 400X240", 4, 4, 1, 1, 1, 1)
}
```

どちらの関数も必須ではありません。トップレベルの文だけを持つ main.wick も正しいプログラムです。fn の外にあるトップレベルの文は、読み込み時に一度だけ実行されます。リソースの作成や状態の初期設定はそこで行います。この二つの関数を書いてファイルを保存し、フォルダを実行すると、濃い青で画面を消去し、組み込みの 8×8 フォントで一行の文字を表示するウィンドウが現れます。ゲームと呼べる最小限の形です。

2.3 変数と最初の動き

何かを動かしてみましょう。変数はすべて let で導入します。型は初期値から推論されるため、明示する機会は少なくなります。

```
let x = 200.0      // num, inferred
let y = 120.0
```

これはトップレベルの let なので、x と y はプログラムのグローバル変数です。フレームをまたいで値を保つので、プレイヤーの位置にちょうど向いています。次に update でキーボードを読み、draw で (x, y) に四角を描きます。

```
let x = 200.0
let y = 120.0

fn update(dt: num) {
  let speed = 120.0
  if lt.key("left")  { x = x - speed * dt }
  if lt.key("right") { x = x + speed * dt }
  if lt.key("up")    { y = y - speed * dt }
  if lt.key("down")  { y = y + speed * dt }
  x = max(8, min(392, x))
  y = max(8, min(232, y))
}

fn draw() {
  lt.clear(0.09, 0.07, 0.16)
  lt.rect(x - 4, y - 4, 8, 8, 1, 0.85, 0.3, 1)
}
```

ここで早くも、いくつか注目すべき点があります。`speed` は `update` 内で `let` を使って宣言しているので、ローカル変数です。その呼び出しの間だけ存在します。前のフレームから経過した秒数 `dt` を掛けると、動きがフレームレートに依存しなくなります。機種が毎秒 60 フレームでも 144 フレームでも、四角は毎秒 120 ピクセル動きます。`max` と `min` は組み込み関数で、名前空間なしで常に使えます。`lt.key` は `bool` を返し、`if` が受け付けるのはこの型だけです。wick に真偽の暗黙変換はありません。`if x` と書いて「x が 0 でなければ」と解釈されることは期待できず、コンパイルもできません。

2.4 関数

関数は `fn` で宣言し、必ずトップレベルに置きます。v0.3 では関数を入れ子にできません。引数の型は必須です。戻り値の型は引数リストの後にコロンで続け、何も返さない関数では省略します。四角を上下に揺らす小さな補助関数を作ります。

```
fn wobble(v: num): num {
  return sin(v * 2) * 6
}
```

`sin` は組み込み関数で、角度をラジアンで受け取ります。`wobble` を使う

には時計が必要なので、トップレベルに `let t = 0.0` を加え、`update` で `t = t + dt` と進めます。描画は次のようになります。

```
lt.rect(x - 4, y - 4 + wobble(t), 8, 8, 1, 0.85, 0.3, 1)
```

覚えておく規則は一つです。関数は、呼び出すより前に宣言する必要があります。コンパイラは一回走査で動作し、その理由は第 7 章で説明します。`update` と `draw` はあなたのコードではなくホストが呼ぶので、両者の順序は問いません。ただし `wobble` は、それを呼び出す行より上になければなりません。

2.5 オプショナル値と得点の保存

ここからが wick らしさの核心です。カウントを増やし、ディスクに保存したいとします。保存は簡単です。

```
lt.save("lamp_best", str(best))
```

`str` は `num` を `str` に変換し、`lt.save` は名前付きの保存スロットにバイトを書き込みます。型システムが力を発揮するのは読み込みです。保存データが存在しない、あるいは読めないかもしれないため、`lt.load_save` は値を保証できません。その戻り値の型は `str` ではなく、*オプショナル値*の文字列 `str?` です。`str` が必要なところに `str?` を使うと、コンパイラが止めます。

```
let s = lt.load_save("lamp_best")   // s: str?
lt.print(s, 4, 4, 1, 1, 1, 1)       // COMPILE ERROR: expected
    str, got str?
```

エラーは `expected str, got str?` です。これは望ましいエラーです。リリースの一か月後に誰かの端末で起きるのではなく、その行を書いた時点で起きます。`str?` を `str` にするには、`nil` の場合にどうするかを指定する必要があります。最も手早い方法は、既定値を与える `??` です。

```
let best = num(lt.load_save("lamp_best") ?? "") ?? 0
```

内側から読みましょう。`lt.load_save("lamp_best")` は `str?` です。最初の `??` は保存データがないときに `""` を使い、`str` にします。`num` は `str` を数値として解析しますが、解析は失敗することがあります。

num("banana") も num("") も数値を作れません。そのため num は num? を返します。二つ目の ?? は解析に失敗したときに 0 を使い、普通の num にします。値が存在しない二つの可能性を一行で処理できました。コンパイラが、どちらも見逃すことを許さなかったためです。序章にあった 0 バイトの保存ファイルのバグが、書けない形になりました。第 4 章で詳しく説明します。

つなぎ合わせましょう。best をトップレベルで宣言し、プレイヤーが Z を押したら増やします。

```
let best = num(lt.load_save("lamp_best") ?? "") ?? 0

fn update(dt: num) {
  // ... movement from before ...
  if lt.pressed("z") {
    best = best + 1
    lt.save("lamp_best", str(best))
  }
}
```

lt.pressed はキーが押されたフレームだけ真になります。一方、lt.key は押している間ずっと真です。ゲームを動かし、Z を何度か押し、終了してから再び実行してみてください。値は保存され、安全に読み直されるので、カウントが残ります。

2.6 リストでランプを持つ

ゲームの名前は *lamp* なので、ランプを加えましょう。四つ用意し、それぞれ位置と点灯・消灯のフラグを持たせます。この最初のチュートリアルでは並行リストを使います。レコードは第 5 章で紹介するため、ここではその代わりとなる*並行リスト*の書き方を使います。複数のリストで同じ添字 i が一つのランプを表します。

```
let lamp_x: list<num> = []
let lamp_z: list<num> = []
let lamp_lit: list<bool> = []
```

空のリストリテラル [] には型推論の材料になる要素がないため、型注釈が必須です。左側の list<num> がそのためにあります。四つのランプでリストを埋めます。

```
push(lamp_x, 3.4)  push(lamp_z, 3.4)  push(lamp_lit, true)
push(lamp_x, -3.4) push(lamp_z, 3.4)  push(lamp_lit, true)
push(lamp_x, -3.4) push(lamp_z, -3.4) push(lamp_lit, true)
push(lamp_x, 3.4)  push(lamp_z, -3.4) push(lamp_lit, true)
```

push は末尾に要素を加え、型も検査します。lamp_lit の要素は bool なので、push(lamp_lit, 3) はコンパイルできません。リストの添字は 0 から始まり、数値範囲で反復します。半開区間 [0, 4) を回る for ループで、すべてのランプを点灯させる関数を作ります。

```
fn reset_lamps() {
  for i in 0..4 { lamp_lit[i] = true }
}
```

for i in a..b は i を a から b の直前まで進めます。終点を含めないのは、添字を 0 から始める方式に合わせるためです。ここでは 0..4 と 0..len(lamp_lit) が同じ意味になります。範囲外の添字を読み書きすると、黙って nil を返すのではなく、行番号付きの実行時エラーになります。すぐ後で、そのエラー画面を見ます。

2.7 一ラウンドを組み立てる

これで、目標のあるゲームを作る材料がそろいました。ランプは時間とともに消え、プレイヤーは近くに立って Z を押すと、最も近いランプを点灯できます。完全な処理は games/showcase_wick/main.wick にあります。ここでは、これまで説明した考え方に絞って更新処理の形を示します。

```
let score = 0

fn nearest_unlit(): num {     // lamp index, or -1 if all lit
  let bi = 0 - 1
  let bd = 0.0
  for i in 0..4 {
    if not lamp_lit[i] {
      let dx = lamp_x[i] - x
      let dz = lamp_z[i] - y
      let d = dx * dx + dz * dz
      if bi < 0 or d < bd { bi = i  bd = d }
```

```
    }
  }
  return bi
}

fn update(dt: num) {
  // ... movement, save on Z ...
  let i = nearest_unlit()
  if i >= 0 {
    if lt.pressed("z") { lamp_lit[i] = true  score = score + 1 }
  }
}
```

ここには後の章につながる点が二つあります。`nearest_unlit` は普通の `num` を返します。添字、または番兵値の `-1` です。このチュートリアルでは複数の戻り値を使わないため、「添字か、何もないか」を返すとき、オプショナル値ではなく通常の範囲外の値で表します。また、最後の二行では `if i >= 0 and lt.pressed("z")` と一つにせず、`if` を入れ子にして条件を単純に保っています。組み合わせた条件をいつ括弧で囲む必要があるかは、第 3 章で扱います。

2.8 開発のループ

二つの機能が、書く・動かす・直すというサイクルを数分から数秒に縮めます。

ホットリロード。 エンジンが動いている間に `main.wick` を保存すると、その場で再コンパイルして読み直します。それまでに作成したリソースは先にすべて解放するので、再読み込みでリークしません。一時間ゲームプレイを調整しても再起動は不要です。定数を変えて保存すると、実行中のゲームが変わります。

エラー画面。 ファイルに誤りがあるとき、エンジンはクラッシュしてターミナルに戻るのではなく、`file:line: message` の形でエラーを画面に描き、待ち続けます。`str?` を `str` として使う、文字列と数値を `+` で結ぶ、呼び出しより下に定義した関数を呼ぶといったコンパイルエラーも、範囲外の添字や失敗した `check()` といった実行時エラーも、同じように表示します。ファイルを直して保存すると、エラー画面からゲーム

がホットリロードされます。実行中のゲームを離れることはほとんどありません。

たとえば `best` の行から `?? 0` を取り除くと、画面には次のように出ます。

```
main.wick:1: expected num, got num?
```

元に戻すまで、ゲームは該当行を示して待ちます。

決定的なスクリーンショット。 `LANTERN_SHOT` にパスを指定すると、エンジンは 60 フレーム目を BMP として保存して終了します。さらに `LANTERN_FIXED_DT=1` を指定すると、フレーム時刻の刻みが固定され、wick の `rand()` も決定的なので、どの機種でもバイト単位で同じ画像が得られます。ゲームプレイ全体を正確に再現できます。第 6 章では、これを基礎にテストの方法を組み立てます。今は「自分の端末では正しく見える」という感覚を、どの端末でも同じ意味を持つ検査にできると覚えておけば十分です。

これで一つのゲームと、それを開発する一連の流れが完成しました。本書の残りでは、本章で例として示した内容を厳密に説明していきます。

第 3 章

言語

本章はリファレンスです。wick v0.3 を網羅的かつ正確に説明します。ソーステキストがトークンになる仕組み、型の種類、変数のスコープ、式の組み立て方と結合の順序、文の種類、制御の流れ、関数の宣言と呼び出しを扱います。チュートリアルでは例で示したところも、本章では規則を明記します。付録 A の文法は、ここでの説明に対応する形式的な定義です。

3.1 字句構造

wick のプログラムは Unicode を表すバイトの列です。字句解析器は左から右、上から下へ読み、トークンの列に変換します。空白と改行はトークンを区切りますが、それ以外の意味はありません。wick には改行に依存する規則もセミコロンもありません。文の終わりを決めるのは改行ではなく文法です。

コメント。 コメントは `//` で始まり、行末まで続きます。ブロックコメントはありません。

```
let x = 10   // this is a comment; it ends at the newline
```

識別子。 識別子は `[A-Za-z_][A-Za-z0-9_]*` に一致します。最初は英字またはアンダースコアで、その後に英字、数字、アンダースコアを任意の数だけ続けます。大文字と小文字を区別します。以下の語は予約されており、識別子には使えません。

```
let  fn  if  elif  else  while  for
in  break  continue  return  true  false  nil
and  or  not  record
```

これがキーワードの一覧です。型名の `num`、`bool`、`str`、`list`、`map` はキーワードではありません。型が必要な文脈でだけ認識されます。自分を混乱させたいなら、変数を `str` と名づけることもできます。

数値。 数値の型は IEEE-754 の倍精度浮動小数点数 `num` 一つだけです。数値リテラルは数字の列で、小数点を一つ含めてもかまいません。たとえば `42`、`3.5`、`0.25` です。16 進の `0xFF` と 2 進の `0b1010` の整数リテラルも、`0xFFFFFFFF` まで受け付けます。接頭辞は大文字でも小文字でも使えます。不正な数字、数字の欠落、アンダースコア、接頭辞の後の小数部、桁あふれはコンパイルエラーです。指数表記は未対応です。字句解析器は範囲演算子に注意しており、`1..5` は数値 `1`、範囲トークン `..`、数値 `5` と解析します。`1.` と `.5` にはしません。小数部のあるリテラルを範囲演算子の隣に置くときは空白が必要ですが、実際の範囲では通常、整数の境界を使います。

文字列。 文字列リテラルは二重引用符で囲み、`\n`（改行）、`\t`（タブ）、`\"`（引用符そのもの）、`\\`（バックスラッシュそのもの）のエスケープを含められます。文字列は不変です。一重引用符の文字列や、複数行の文字列リテラルはありません。

```
let a = "tenzin"
let b = "line one\nline two"
let c = "she said \"hi\""
```

演算子と区切り記号。 字句解析器は、複数文字のトークン `..`（範囲）、`??`（nil 合体）、`==`、`!=`、`<=`、`>=` を、できるだけ長く一致させて認識します。一文字のトークンは、算術・比較演算子 `+ - * / % < >`、代入の `=`、オプショナル値の記号 `?`、メンバーのドット `.`、グループ化と区切りの `( ) [ ] { } , :` です。それ以外の文字は字句エラーになり、`unexpected character 'x'` と報告されます。

3.2 型

wick の式にはすべて静的な型があり、コンパイル時に確定します。スカラー型、コンテナ型、オプショナル型、名前付きレコード型があります。

型	値	備考
num	IEEE-754 倍精度	唯一の数値型
bool	true, false	条件が受け付ける唯一の型
str	不変のテキスト	+ は str 同士だけを連結
list<T>	順序付き、添字は 0 から	スカラーまたは平坦なレコードの要素
map<T>	文字列をキーに持つ	スカラーまたは平坦なレコードの値
T?	T または nil	中心機能であるオプショナル値

list と map の要素型には、num、bool、str、または宣言済みの平坦なレコード型を使えます。v0.3 には入れ子のコンテナがありません。list<list<num>> や map<list<str>> は存在せず、指定すると container elements must be scalar or record types というエラーになります。第 5 章で添字計算による回避策を示します。

レコード。 トップレベルの record で、num、bool、str の名前付きフィールドを宣言します。すべてのフィールド名を指定して値を作り、ドットでフィールドを参照・代入します。メソッドや入れ子のフィールドはありません。

```
record Register { value: num, name: str }
let a = Register { value: 0, name: "A" }
a.value = 0xFF
check(a.value == 255, "record field")
```

nil とオプショナル値。 nil は一般的な型の値ではありません。オプショナル型だけに属します。T? の式は nil または T を持てますが、型が分からない単独の nil は書けません。let x = nil はコンパイルできません。何の値が「存在しないかもしれない」のか推論できず、コンパイラは can't infer a type from nil; annotate: let x: str? = nil と伝えます。型注釈を付ければ問題ありません。

```
let x: str? = nil      // x is str?, currently nil
```

第 4 章全体でオプショナル値を扱います。ここでは、T? はコンパイラが追跡する正式な型であり、T が必要な場所では T? を使えないことだけ押さえます。

void。 戻り値の型を省略して宣言した関数の戻り値型は `void` です。`void` は値ではありません。void 関数の呼び出しは式ではなく文であり、`let` の右辺や、より大きな式の中には置けません。試みると `can't assign a void expression` と報告されます。

3.3 変数とスコープ

変数はすべて `let` で導入します。名前を新しく作る方法は、これしかありません。特に、宣言していない名前への代入は暗黙の宣言ではなくコンパイルエラーです。この一つの規則で、Lua につきまとう「タイプミスがグローバル変数を作る」バグをまとめて取り除きます。

```
let x = 10          // num, inferred from 10
let y: num = 10     // the same, written out
let s = "hi"        // str
```

型は初期値から推論するか、コロンの後に明示できます。空のコンテナリテラルや単独の `nil` など、推論の材料がないときは明示しなければなりません。分かりやすさのために、いつでも書いてかまいません。初期値と型注釈が両方あるときは一致が必要です。

グローバル変数とローカル変数。 ファイルのトップレベルの `let` は、プログラムのグローバル変数を作ります。ファイルの読み込み時に一度初期化され、すべてのフレームを通じてプログラムの終了まで存続します。関数内の `let` はローカル変数を作ります。呼び出しのスタックフレーム内に存在し、呼び出しが戻るとなくなります。そのため、プレイヤーの位置や得点など、フレームをまたぐ状態はトップレベルの `let` に置き、一時的な値はローカル変数に置きます。

ブロックとシャドーイング。 変数のスコープは、宣言された波括弧で囲むブロックと、その内側のブロックです。内側のブロックで同名の `let` を新しく宣言すると、外側の名前を隠すことができます。内側のブロックが終わるまで、内側の名前が外側の名前を隠します。*同じスコープでの再宣言はエラーです。*

```
let n = 1
if true {
  let n = 2      // ok: shadows the outer n inside this block
}
```

```
let n = 3        // ERROR: 'n' already declared in this scope
```

最後の行のエラーは `'n' already declared` です。既存の変数を変更するには、もう一度 `let` を使うのではなく、代入します。

代入。 代入は一つの `=` を使い、代入先の名前は宣言済みでなければなりません。未宣言の名前を読むと `unknown variable 'x' (declare it with let)`、代入すると `unknown variable 'x' --- wick has no implicit globals; use let` と報告されます。代入する値の型は変数と互換性が必要です。互換性の規則は後で説明します。

3.4 式と優先順位

式は、リテラル、変数、呼び出し、添字、演算子を組み合わせます。演算子は次の優先順位で結合します。表は結合が最も弱いもの、つまり最後に評価されるものから、最も強いものへ並べています。二項演算子はすべて左結合です。

順位	演算子	意味
1	`or`	論理和（短絡評価）
2	`and`	論理積（短絡評価）
3	`??`	nil 合体
4	`==` `!=`	等価、不等価
5	`<` `<=` `>` `>=`	大小比較
6	`+` `-`	加算・連結、減算
7	`*` `/` `%`	乗算、除算、剰余
8	単項 `-`, `not`	符号反転、論理否定
9	`f(x)`, `a[i]`	呼び出し、添字

したがって `a or b and c` は `a or (b and c)` と解析され、`1 + 2 * 3` は算術の慣例どおり `1 + (2*3)` となります。初めて使う人が戸惑いやすいのは、`??` が比較や算術より弱く、`and`・`or` より強く結合することです。`num(s) ?? 0` に括弧が不要なのはこのためです。合体演算が `num(s)` 全体にかかり、結果はそのまま使える普通の `num` になります。

算術。 `+ - * / %` は `num` を受け取り、`num` を返します。両方のオペランドは非オプショナルの `num` でなければなりません。オプショナル値は中の値を取り出すまでエラーです。ゼロ除算は Lua と同じく IEEE-754 に従い、例外停止ではなく無限大や NaN を生成します。

+ の二つの役割。 + は両方が num なら数値の加算、両方が str なら文字列の連結です。混在は許しません。暗黙の変換はないので、一方が str、他方が num の "score " + 5 はコンパイルできません。エラーは修正方法も示します。'+' needs num+num or str+str (use str(x))。"score " + str(5) と書きます。これは意図した設計です。数値を黙って文字列にすることは多くのバグの原因になるため、wick は意図を明示することを求めます。

比較と等価性。 大小比較の < <= > >= は num のオペランドを必要とし、bool を返します。等価の == と不等価の != は同じ型の二つの値を比べ、bool を返します。str ではオブジェクトの同一性ではなく内容を比べます。異なる型の比較は '==' operands must have the same type というエラーです。ただし、オプショナル値と nil の比較は認められます。第 4 章で説明する型の絞り込みの書き方です。*非*オプショナル値と nil の比較は、逆に comparing non-optional to nil というエラーです。非オプショナル値は nil にならず、その検査は無意味だからです。

論理演算子。 and と or は bool を受け取り、bool を返し、短絡評価を行います。a and b は a が偽なら b を評価せず、a or b は a が真なら b を評価しません。not は bool を受け取り、反転します。真偽の暗黙変換はないため、a が数値のとき a and b とは書けません。意図する bool を返す比較を書きます。

nil 合体演算子 ??。 a ?? b の a はオプショナル型 T? である必要があります。a が nil でなければ a の中の値を取り出し、そうでなければ b を評価します。既定値 b は元の型 T でなければならず、式全体も非オプショナルの T になります。右辺は左辺が nil のときだけ評価します。非オプショナルの左辺に ?? を使うと、合体させる必要がないため '??' left side must be an optional というエラーになります。

組み合わせた条件と括弧。 型を絞り込む比較 x != nil を and や or と組み合わせると、コンパイラは括弧を求めます。読む人と絞り込み解析が、構造を曖昧さなく把握するためです。メッセージは parenthesize this condition: (a != b) and ... です。括弧を加えれば解決します。

```
if (s != nil) and ready { }    // parenthesised as the compiler asks
```

3.5 文

wick のプログラム本体は文の列です。文には次の形があります。

```
let x = expr            // declaration (with optional : type)
x = expr                // assignment to a declared variable
xs[i] = expr            // element assignment (list index or map
    key)
f(a, b)                 // call statement; a non-void result is
    discarded
if c { } elif c2 { } else { }
while c { }
for i in a..b { }       // i: num, from a to b-1
break                   // exit the innermost loop
continue                // next iteration of the innermost loop
return expr             // or bare `return` in a void function
```

ブロックは常に波括弧を使います。一文だけだからといって省略する形はありません。文として使った呼び出しは、結果があれば破棄します。副作用のために関数を呼ぶときの方法です。要素への代入はリストの添字かマップのキーに対して行い、対象はすでにリストまたはマップである必要があります。他の値に添字を使うと `only lists and maps can be indexed` となります。

3.6 制御フロー

条件分岐。 `if`、任意の `elif` の連なり、任意の `else` で条件分岐を構成します。条件はすべて `bool` である必要があります。真偽の暗黙変換はないので、`if 1 { }` や `if name { }` はコンパイルできません。エラーは `if condition must be bool (wick has no truthiness)` と規則を直接示します。`if x > 0`、`if s != ""` のように、意図する比較を書きます。

```
if score > best {
  best = score
} elif score == best {
  // tie
} else {
  // nothing
}
```

while。 while は bool の条件が成り立つ間、本体を繰り返します。条件は各反復の前に検査します。

```
let i = 0
while i < 4 {
  i = i + 1
}
```

for の範囲。 for i in a..b は、ループ変数 i を半開区間 [a, b)、つまり a から b の直前まで反復します。両端は num の式で、一度だけ評価します。ループ変数は新しい num のローカル変数で、スコープはループ本体です。v0.3 の反復形式はこれだけで、for x in list はありません。リストをたどるには添字を反復します。

```
for i in 0..len(xs) {
  lt.print(str(xs[i]), 4, 4 + i * 10, 1, 1, 1, 1)
}
```

終点を含めないのは、0 からの添字に意図的に合わせています。0..len(xs) は有効な添字 0 から len(xs) - 1 だけを正確に訪れるので、端の数を一つ間違える余地がありません。

break と continue。 break は直近の外側のループを抜け、continue はそのループの次の反復に進みます。どちらもループ外ではエラーで、break outside a loop、continue outside a loop と報告されます。

3.7 関数

関数は fn でトップレベルにだけ宣言します。v0.3 では関数の入れ子、クロージャ、第一級の関数値はありません。入れ子の fn は fn declarations can't nest と報告されます。

```
fn dist(ax: num, az: num, bx: num, bz: num): num {
  let dx = bx - ax
  let dz = bz - az
  return sqrt(dx * dx + dz * dz)
}
```

引数と戻り値の型。 すべての引数には型を明示します。引数の型推論はありません。戻り値の型がある場合は、引数リストの後にコロンで続けます。省略すれば `void` です。`void` の関数は単独の `return` で早期終了できますが、`return expr` は使えません。`void function can't return a value` というエラーです。戻り値の型を宣言した関数は、値を返す経路で、その型の `return expr` を使う必要があり、単独の `return` は使えません。`this function must return T` と報告されます。

使用前の宣言。 関数は、最初の呼び出しより上に宣言する必要があります。コンパイラは一回走査で、呼び出しに到達した時点で解決するため、ファイルの後ろにある関数を呼ぶと `unknown function 'f' (wick requires declare-before-use)` となります。定義を上に移せば直ります。ホストが呼ぶ二つの関数 `update` と `draw` は、互いの順序を問いません。毎フレーム名前で検索するのは、あなたのコードではなくホストだからです。

再帰。 関数本体の中では自身の名前がスコープ内にあるため、自分自身を呼び出せます。

```
fn fib(k: num): num {
  if k < 2 { return k }
  return fib(k - 1) + fib(k - 2)
}
```

呼び出しの検査。 呼び出しは完全に型検査されます。引数の数を間違えると `f() needs at least N arguments` または `f() takes N arguments` と報告されます。型を間違えると、たとえば `f() argument 3: expected num, got str` のように、関数名と引数の位置を示します。これは自作の関数にも、型付きシグネチャを登録するエンジン呼び出しにも同じように適用されます（第 6 章）。

戻り値を返す経路の網羅。 v0.3 は、非 void 関数のすべての経路が値を返すかどうかをまだ検証しません。戻り値の型を宣言した関数で末尾まで到達すると、呼び出しは `nil` を返します。この挙動に依存してはいけません。第 8 章の制約にも記載しています。守るべき習慣は簡単です。値を返す関数の最後の文を `return` にします。

3.8 代入の互換性

ある型を別の型が必要な場所に格納できるかどうかの規則は短いので、まとめて示します。

- `T?` が必要な場所に `T` を格納できます。オプショナル型への拡張はコストがなく、常に安全です。
- `T` が必要な場所に `T?` は格納できません。先に `??` または型の絞り込みで中の値を取り出します。
- コンテナは不変です。`list<num>` は `num` の要素だけを受け付け、`list<num>` は `list<num?>` など別の型ではありません。要素型はコンテナ型の一部で、正確に一致する必要があります。
- `void` は値ではないため、まったく格納できません。

この四つの規則と、次章のオプショナル値の扱いが、wick の型検査のすべてです。`T` から `T?` への拡張以外の部分型関係も、暗黙の型変換も、組み込みの `list` と `map` 以外のジェネリクスもありません。仕組みは意図的に小さくしてあります。頭の中に収まる小ささこそ、この言語全体の狙いです。

第 4 章

オプショナル値

wick に存在理由が一つだけあるとすれば、オプショナル値です。静的な型、真偽の暗黙変換の禁止、コンパイル時のエンジン呼び出し検査は、すべて一つの主張を支えるものです。「この値は存在しないかもしれない」という事実は、原因から三つ先の関数で突然現れる実行時の問題ではなく、コンパイラが知り、プログラマが処理すべき事実である。本章ではオプショナル値を詳しく説明し、それを譲れない要件にしたバグの話を紹介します。

4.1 Lua で起きた問題

失敗から始めましょう。この機能は、その失敗の形にぴったり合わせて設計したからです。Lantern Night は保存ファイルに最高得点を記録します。Lua 版は次のように読み込んでいました。

```
local best = tonumber(lt.load_save("lanternnight_best") or "0")
```

どの部分も妥当に見えます。`lt.load_save` は保存した文字列を返し、保存データがなければ `nil` を返します。`or "0"` は `nil` のときの既定値を与え、`tonumber` は文字列を数値として解析します。正しく読め、レビューを通り、出荷されました。

その後、プレイヤーが保存中にゲームを強制終了しました。保存はディスクへの書き込みです。まずい瞬間に中断すると、ファイルだけが作られ、中身が空の 0 バイトになります。次の起動時、`lt.load_save` はそのファイルを開き、正常に読み取り、`nil` ではなく空文字列 `""` を返しました。`""` を手に、式をもう一度たどってみましょう。

- `lt.load_save(...)` は `""` です。`nil` ではなく、実在する文字列です。
- Lua では空文字列が真として扱われるので、`"" or "0"` は `""` です。代替値は使われません。

- 空文字列は数値ではないため、`tonumber("")` は `nil` です。
- `local best = nil` となり、以後の `best` の比較、算術、数値の描画は、すべて `nil` に対して行われます。

ゲームは最初のフレームでクラッシュしました。保存や読み込みの場所ではなく、その先で、`nil` が初めて許容されない演算に出会ったところです。スタックトレースは、何の落ち度もない行を指していました。この現象には Lua の四つの判断が重なる必要がありました。空文字列を真とすること、空ファイルの読み込みが `nil` ではなく `""` を返すこと、解析失敗を `nil` で表すこと、`nil` の伝播を止める静的検査がないことです。それぞれ単独では合理的です。それでも組み合わさると、ディスク書き込みの中断が、コードを読んでも見抜けない最初のフレームのクラッシュに変わりました。

4.2 バグを書けなくする型

wick の答えは型です。値を生成できない可能性のある操作は、その値を直接返すのではなく、`T?` と書く*オプショナル値*を返します。これは `T` または `nil` です。コンパイラは `T?` を `T` として使うことを許しません。

```
let s = lt.load_save("hi")        // s: str?
lt.print(s, 4, 4, 1, 1, 1, 1)     // COMPILE ERROR: expected str,
    got str?
```

二行目はコンパイルできません。エラーは `expected str, got str?` で、その行を書いた瞬間に、プレイヤーの端末ではなく自分の端末で現れます。`str` を必要とする `lt.print` に `str?` が届く経路はありません。`str?` から `str` に進むには、値が `nil` のときどうするかをプログラム内で示す必要があります。wick にはその方法が二つあります。

4.3 ?? で中の値を取り出す

nil 合体演算子 `??` は、`nil` の場合に既定値を与えて、オプショナル値から中の値を取り出します。`a ?? b` は `a` が `nil` でなければ `a`、そうでなければ `b` になります。結果の型は非オプショナルの元の型です。

```
let text = s ?? "no save yet"     // text: str
let n = num("maybe") ?? 0         // n: num  (num parses str ->
    num?)
```

既定値は元の型に一致しなければなりません。s は str?、既定値は str なので s ?? "..." は使えます。一致しないと '??' default must be str と報告されます。str の部分は元の型に応じて変わります。右辺は左辺が nil のときだけ遅延評価されるので、そこで処理を行っても安全です。また、?? の左辺は実際にオプショナル値でなければなりません。nil になり得ない値に使うと、合体させる必要がないため '??' left side must be an optional というエラーになります。

実例に戻りましょう。あの致命的な Lua の行を wick にすると、次のようになります。

```
let best = num(lt.load_save("lanternnight_wick_best") ?? "") ?? 0
```

Lua 版をクラッシュさせた 0 バイトのファイルをたどります。

lt.load_save(...) は str? を返します。ファイルがなくて nil なのか、空ファイルで "" なのかは、その後の処理に影響しません。最初の ?? がどちらも str にするからです。nil は "" になり、"" はすでに "" です。次の num("") は num? で、空文字列は数値ではないので nil です。二つ目の ?? が受け止め、0 を返します。保存ファイルが存在するか、形式が正しいか、中身が空かによらず、結果は必ず普通の num です。バグを直したというより、バグを書けなくしたのです。どちらかの ?? を消すと、コンパイルできません。

この違いを身につけることが大切です。Lua では安全な版とバグのある版がほぼ同じに見え、コンパイラは両方を受け付けます。違いが現れるのは、本番環境で特定のまれな入力が来たときだけです。wick では ?? が欠けたバグのある版はコンパイルエラーで、安全な版だけがビルドできます。型システムは、バグの発見をプレイヤーの最初のフレームから、作者の最初の保存へ移したのです。

4.4 if x != nil で型を絞り込む

オプショナル値を扱う二つ目の方法は、型を絞り込むことです。nil と比較し、検査を通過したブロック内で普通の T として使います。

```
if s != nil {
  // inside this block, s has type str, not str?
  lt.print(s, 4, 4, 1, 1, 1, 1)
}
```

`s != nil` は二つの役割を果たします。`bool` の条件であると同時に、保護されたブロック内では `s` が `nil` でないことを型検査器に伝えるので、その中の型は `str` になります。読む人が直感的に行う「nil でないと確かめたので、ここでは文字列だ」という判断を、形式化し強制しています。

値を使っていくつかの処理を行いたく、既定値では扱いにくいときや、`nil` の場合に独自の処理があるときは、絞り込みが適しています。

```
let saved = lt.load_save("name")
if saved != nil {
  lt.print("WELCOME BACK " + saved, 4, 4, 1, 1, 1, 1)
} else {
  lt.print("NEW PLAYER", 4, 4, 1, 1, 1, 1)
}
```

絞り込みの制約。 v0.3 の絞り込みはローカル変数に適用されます。モジュールのグローバル変数は、検査と使用の間に別の関数や後のフレームのコードが変更し得るため、`if` では絞り込まれません。グローバルのオプショナル値には `??` を使うか、先にローカル変数へコピーしてから絞り込みます。ブロック内で、絞り込んだ変数に新しいオプショナル値を代入すると、絞り込みは解除されます。保証があるのは検査した値だけです。また、v0.3 では `and`・`or` の連なりを通じて絞り込みが伝わりません。組み合わせた条件にコンパイラが括弧を求める理由です（第 3 章）。迷ったら入れ子の `if` で絞り込むか、ローカル変数へコピーします。いずれも第 8 章に記載した既知の v0.3 の制約で、バグではありません。

4.5 オプショナル値を返す操作

オプショナル値は適当に付け足すものではありません。実際に失敗し得る操作から生まれ、コンパイラがそこから伝播させます。v0.3 では以下が該当します。

式	型	`nil` になる場合
`lt.load_save(k)`	`str?`	読み込める保存データがない
`num(s)`	`num?`	`s` が数値でない（`""` も含む）
`pop(xs)`	`T?`	リストが空
`m[key]`	`T?`	キーが存在しない
`env(name)`	`str?`	変数が設定されていない

いずれも他の言語なら、空文字列、偽、nil、「見つからない」を表す特殊な値など、確認を忘れやすい番兵値を受け取るところです。wick は失敗を型で表すため、確認を忘れることができません。忘れた確認はコンパイルエラーになります。最も分かりやすいのはマップの検索です。`m[key]` は、キーの存在を確信していても必ず `T?` です。型システムには、あなたの確信は分からないからです。キーがない場合を呼び出し箇所で必ず意識することになり、`??` なら三文字で処理できます。

```
let hp = stats["health"] ?? 100
```

4.6 型システムの中のオプショナル値

オプショナル値と言語の他の部分をつなぐ規則を、第 3 章から、オプショナル値の側から見直してまとめます。

- **拡張はコストがありません。** `T?` が必要な場所には `T` を使えます。「必ずある」から「ないかもしれない」へ移っても、安全性は失われません。
- **中の値を取り出す必要があります。** `T` が必要な場所に `T?` は使えません。移れるのは `??` と絞り込みだけで、どちらも `nil` を扱う必要があります。
- **オプショナル値は入れ子になりません。** `T??` はありません。オプショナル値は `T` または `nil`、それだけです。
- **`nil` には型が必要です。**単独の `nil` からは型を推論できません。`let x: str? = nil` と書けば指定できます。`nil` は、特定のオプショナル型の空の場合としてだけ意味を持ちます。
- **非オプショナル値と `nil` の比較はエラーです。** `x` が普通の `str` なら、`x` は `nil` になり得ないので `x != nil` は無意味なコードです。コンパイラは必ず成功する検査を許すのではなく、`comparing non-optional to nil` と伝えます。

4.7 実際の書き方

日々 wick を使うと、動的な言語が要求する防御的な足場を書かなくなります。足場が守っていた事実を、コンパイラが追跡するからです。読

み込みと解析を毎回 `if x ~= nil` の連なりで囲む必要はありません。値の入り口で一度 `??` を書けば、その先は裏切らない普通の `T` です。オプショナル値は余計な負担ではありません。どの値が `nil` かもしれないか覚えておく、手作業で、忘れやすく、レビューしても見逃す負担を取り除きます。代わりにコンパイラが覚え、忘れることを拒みます。

0 バイトの保存ファイルは一つのバグです。しかし「失敗し得る操作の失敗を普通の値で表せるため、確認を省けてしまう」という種類のバグは膨大です。wick はこの種類をなくすために作られました。wick プログラムにある一つ一つの `T?` は、その種類の一例を処理したという小さな証明です。

第 5 章

コレクション

wick には、リストとマップの二つのコンテナ型があります。また、まだ持っていない構造体の代わりとなる並行リストという書き方があります。本章ではこの三つを扱い、0 からの添字に合う反復のパターンと、失敗し得る検索をすべてオプショナル値にする意図的な選択を説明します。

5.1 リスト

リストは型を持つ順序付きの列で、添字は 0 から始まります。要素型は三つのスカラー型のいずれかです。リストリテラルは内容から要素型を推論し、空のリテラルには型注釈が必要です。

```
let xs = [1, 2, 3]            // list<num>, element type inferred
let names = ["a", "b"]        // list<str>
let flags: list<bool> = []    // empty literal REQUIRES the
    annotation
```

空リストの注釈は任意の心遣いではありません。`[]` には推論の材料がなく、コンパイラは `empty [] needs a type: let xs: list<num> = []` と伝えます。リストは同じ型の要素だけを持つので、型が混在するリテラルもコンパイルできません。`list elements must share one type` と報告され、`[1, "a"]` は拒否されます。

リストの操作。 基本操作は組み込み関数で、静的な要素型に応じて処理されます。

```
push(xs, 4)                   // append; type-checked against the
    element type
let last = pop(xs) ?? 0       // remove and return the last; T?
    (nil if empty)
xs[0] = 10                    // index assignment
```

```
let head = xs[0]              // index read
let n = len(xs)               // number of elements
```

push は要素型に一致する値を末尾に加えます。push(flags, 3) はコンパイルできません。pop は最後の要素を取り除いて返しますが、リストは空かもしれないため、結果はオプショナル値 T? です。取り出すものがなければ nil になります。他のオプショナル値と同様、通常は ?? default で扱います。len は要素数を num として返します。

添字は実行時に検査します。 [0, len) の範囲外を読み書きすると、未定義の動作や黙って返される nil ではなく、*実行時*エラーになります。そのフレームを止め、エラー画面に行番号と list index N out of range (len M) を表示します。静的な型システムには、一般に添字の値までは分からないので、この検査は動的なままです。ただし、メモリの安全性の穴ではなく、正確なメッセージ付きの検査です。

要素型。 リストの要素は num、bool、str です。v0.3 にリストのリストはなく、list<list<num>> は container elements must be num, bool, or str と報告されます。本当に格子や不規則な構造が必要なときの回避策は、平坦なリストに対する添字計算です。本章の最後で示します。

5.2 マップ

マップは文字列をキーとし、値の型を持つコレクションです。キーは常に str、値は三つのスカラー型のいずれかです。

```
let scores = ["ana": 3, "bo": 5]   // map<num>
scores["cid"] = 1                  // insert or update
let s = scores["dee"] ?? 0         // get is ALWAYS T? --
    missing key is nil
let n = len(scores)                // number of entries
```

マップリテラルは文字列リテラルのキーと値を組にします。リテラル内のキーは計算式ではなく文字列リテラルでなければなりません。map keys must be str literals という規則です。計算したキーで挿入するには、m[k] = v の添字代入を使います。リテラル内の値は、リストの要素と同様に、同じ型である必要があります。

検索結果は必ずオプショナル値です。 マップで最も大切な事実は、`m[key]` の型が必ず `T?` であることです。直前に挿入したキーでも同じです。型システムはキーの存在を証明できないため、どの検索も存在しない可能性があると扱い、`T?` を渡します。これは意図的な設計です。誰もが忘れるキー不在の場合を呼び出し箇所で必ず意識し、`??` なら三文字で処理できます。型が仮定を許さないため、「さっきはあったのに」と驚くことはありません。

```
scores["ana"] = 10
let a = scores["ana"] ?? 0    // still T?; the ?? is required,
    not optional
```

5.3 レコードとレコードのリスト

ランプ、レジスタ、回路部品は、論理的には一つの値です。Wick 0.2 からは、平坦なレコードが、その値に名前付きで検査されるフィールドを与えます。

```
record Lamp { x: num, z: num, lit: bool }
let lamps: list<Lamp> = []
push(lamps, Lamp { x: 3.4, z: 3.4, lit: true })
push(lamps, Lamp { x: -3.4, z: 3.4, lit: true })
lamps[0].lit = false
check(lamps[1].x == -3.4, "named field")
fn relight(i: num) { lamps[i].lit = true }
```

レコードは使用前に宣言します。構築時にはすべてのフィールド名を指定し、欠落、重複、未知のフィールド、誤った型はエラーです。フィールドは `num`、`bool`、`str` に限定され、レコードにはメソッドや入れ子のコンテナがありません。レコードはヒープ上のオブジェクトです。別の変数へ代入すると共有されるため、フィールドの変更はどちらの参照からも見えます。独立したコピーが必要なときは、新しい値を構築します。

古いチュートリアルは元のゲームと比較できるように、スカラーの並行リストを使っています。その表現も引き続き使えますが、レコードなら別々のフィールド用リストを同じ長さに保つ必要がなくなります。入れ子のデータには、今でも平坦な ID や別々のリストが必要です。第 9 章では 8 ビットのレジスタにレコードを使います。

5.4 反復のパターン

コレクションを回るループ形式は一つです。添字を反復し、その添字で要素にアクセスします。

```
for i in 0..len(xs) {
  use(xs[i])
}
```

範囲は終点を含まず、リストの添字は 0 からなので、`0..len(xs)` は有効な添字だけを正確に訪れます。一つずれる誤りがありません。v0.3 に `for x in list` はなく、添字による形式だけです。その利点は、必要なときに `i` が手元にあることです。並行リストでは常に必要で、最も近い要素を探す問題では戻り値として必要です。

添字を返す検索。 よくある形は、「ある条件に合う要素を探し、なければないと伝える」です。複数の戻り値も `for x in list` もないため、そこまでの最良の添字と、「なし」を示す番兵値を持つループを書きます。

```
fn nearest_unlit(): num {        // returns a lamp index, or -1
  let bi = 0 - 1
  let bd = 0.0
  for i in 0..len(lamp_x) {
    if not lamp_lit[i] {
      let dx = lamp_x[i] - px
      let dz = lamp_z[i] - pz
      let d = dx * dx + dz * dz
      if bi < 0 or d < bd {
        bi = i
        bd = d
      }
    }
  }
  return bi
}
```

呼び出し側は結果を使う前に番兵値を確認します。

```
let i = nearest_unlit()
if i >= 0 {
  // i is a valid index
```

```
}
```

初期値の `bi` は `-1` ではなく `0 - 1` と書いています。単項マイナスはあるため、どちらも合法です。このゲームのコードでは、たまたま減算の形にしています。`-1` はオプショナル値ではなく、普通の `num` の番兵値です。呼び出し側へ `num?` を伝えていくより `i >= 0` の検査の方が軽いためです。これは、言語に欠けている複数の戻り値の代用として定番の書き方です。

5.5 入れ子のコンテナを使わない入れ子データ

タイルの格子など、本当に二次元の構造が必要でも、`list<list<num>>` は使えません。その場合は平坦化します。一つのリストに行優先で格納し、添字を計算します。

```
let W = 16
let H = 12
let tiles: list<num> = []
for i in 0..W * H { push(tiles, 0) }

fn get(tx: num, ty: num): num {
  return tiles[ty * W + tx]
}

fn set(tx: num, ty: num, v: num) {
  tiles[ty * W + tx] = v
}
```

システム言語で生のバッファを扱うときと同じ工夫です。実行時の範囲検査とも問題なく組み合わせられます。範囲外の `ty * W + tx` は、他の範囲外の添字と同じように検出されます。入れ子のコンテナより記述は長くなりますが、対応するまでは、これが v0.3 の率直な答えです。

第 6 章

エンジンインターフェース

wick のプログラムは `lt` 名前空間を通じて lantern エンジンとやり取りします。すべての `lt.*` 呼び出しには型付きシグネチャが登録され、コンパイラが検査します。引数の数や型が間違っていれば、自作の関数と同様、呼び出し名と引数の位置を示すコンパイルエラーになります。本章では、ゲーム全体を構成するフレームの契約を述べ、API を分野別に説明します。その後、エンジンと言語を一緒に設計して実現した、決定性と検証可能性の二つの性質を扱います。

6.1 フレームの契約

エンジンは名前で探した二つの関数を、毎秒 60 フレームで、各フレームに一度ずつ呼び、ゲームを動かします。

```
fn update(dt: num) { }   // simulation; dt is seconds since the
    last frame
fn draw() { }            // rendering; 3D calls first, 2D
    composites on top
```

`update(dt)` はシミュレーションを進めます。引数 `dt` は経過秒数で、*上限を 0.1 に制限*しています。ブレークポイントや遅い読み込みなどで長く止まっても、一度の巨大なステップでプレイヤーが壁をすり抜けることを防ぎます。`draw()` はフレームを描画します。3D 呼び出しを先に行い、2D 呼び出しをその上へ合成するので、`lt.print` や `lt.rect` で描く HUD は常にシーンの上に出ます。

どちらの関数も必須ではありません。両方ない `main.wick` も合法で、トップレベルの文を一度実行し、静止したフレームを表示します。`fn` の外のトップレベルの文は、読み込み時に正確に一度だけ実行します。メッシュを作り、テクスチャや音を読み込み、状態を初期化する場所です。順序は固定なので、覚えておきましょう。トップレベルを一度、その後は毎フレーム `update` と `draw`、最後にフレーム境界で回収を行います

(第 7 章)。

6.2 フレームと画面

呼び出し	備考
`lt.W lt.H`	コンパイル時の定数：400、240
`lt.clear(r, g, b)`	色と深度を消去。`draw()` の最初に呼ぶ
`lt.time(): num`	起動後の秒数（`LANTERN_FIXED_DT` では固定刻み）
`lt.screenshot(path: str)`	400×240 のフレームを BMP として保存
`lt.quit()`	正常終了を要求
`lt.escape_quits(enable: bool)`	既定では Escape で終了。ポーズメニュー用に無効化可能

`lt.W` と `lt.H` は呼び出しではなく数値定数です。コンパイル時に 400 と 240 になるので、`lt.W / 2` はレイアウトに自由に使える定数式です。画面は常に 400×240 で、エンジンが表示画像をウィンドウに合わせて拡大・縮小します。

6.3 3D シーン

呼び出し	備考
`lt.camera(ex,ey,ez, tx,ty,tz, fov=55)`	視点の位置と注視点
`lt.light(dx,dy,dz, ambient=0.35)`	平行光源
`lt.point_light(i, x,y,z, radius, r=1,g=1,b=1)`	スロット 0–3。`radius <= 0` で無効化
`lt.fog(start, end, r,g,b)`	視点からの深度に線形。`end <= start` で無効化

`fov`、`ambient` のように、末尾にあり既定値を持つ引数は省略できます。コンパイラが既定値を補うので、ネイティブ関数には常にすべての引数が届きます。点光源には 0 から 3 の四つのスロットがあります。半径を 0 以下に設定すると、そのスロットを無効化します。Lantern Night はこの方法でランプを消します。

```
if lamp_lit[i] {
```

```
  lt.point_light(i, lamp_x[i], 0.8, lamp_z[i], 3.6, 1.0, 0.70,
    0.28)
} else {
  lt.point_light(i, 0, 0, 0, 0, 0, 0, 0)   // radius 0: off
}
```

6.4 メッシュ

呼び出し	備考
`lt.cube(): num`	単位立方体
`lt.plane(segs=1): num`	分割した単位 XZ 平面
`lt.sphere(seg=12)`, `lt.cylinder(seg=16)`, `lt.cone(seg=16)`	単位プリミティブ
`lt.mesh(verts: list<num>): num`	独自メッシュ。頂点ごとに浮動小数点数 12 個
`lt.load_mesh(path: str): num`	Wavefront OBJ（法線がなければ面法線を使用）
`lt.draw(m, x,y,z, rx,ry,rz, sx,sy,sz, r,g,b, tex=-1)`	グーロー照明で描画（回転・拡縮・色に既定値）
`lt.draw_lerp(a, b, t, x,y,z, …, tex=-1)`	頂点数が同じ二つのメッシュ間のキーフレーム補間
`lt.billboard(tex, x,y,z, w,h, u0,v0,u1,v1)`	カメラを向く四角形（スプライトのキャラクター）
`lt.shadow(x,y,z, radius, alpha=0.35)`	接地感を出す簡易の影

メッシュの構築関数はすべて `num` のハンドルを返します。ハンドルはエンジンが所有する不透明な整数なので、メッシュ型はありません。`lt.mesh` は、頂点ごとに位置 3 個、法線 3 個、テクスチャ座標 2 個、色 4 個、合計 12 個の浮動小数点数を持つ平坦な `list<num>` を受け取ります。第 5 章の平坦なリストの扱い方によく合います。`lt.draw` は末尾に多数の既定引数を持つため、よく使う場合は短く書けます。

```
let ground = lt.plane(12)
lt.draw(ground, 0, 0, 0, 0, 0, 0, 15, 1, 15, 0.42, 0.40, 0.40)
```

6.5 2D

呼び出し	備考
`lt.rect(x,y,w,h, r,g,b, a=1)`	塗りつぶし、アルファ合成
`lt.load_texture(path: str): num`	BMP。マゼンタ (255,0,255) は透明
`lt.sprite(tex, x,y, sx=1,sy=1)`	左上を基準点に配置
`lt.sprite_ex(tex, cx,cy, sx=1,sy=1, rot=0, r=1,g=1,b=1,a=1)`	中心基準、回転、着色。負の倍率で反転
`lt.sprite_uv(tex, x,y,w,h, u0,v0,u1,v1)`	アトラス内の部分矩形（タイルマップ）
`lt.print(text: str, x,y, r=1,g=1,b=1,a=1)`	組み込み 8×8 フォント。`\n` に対応

`lt.print` は組み込みの 8×8 ピクセルのフォントで描画し、`\n` を解釈します。`+` は `str` と `str` しか連結しないため、HUD の行を作るには `str` で数値を明示的に変換します。Lua の `string.format` と違い、整数を末尾の小数なしですっきり表示します。

```
lt.print("LAMPS RELIT: " + str(score), 4, 3, 1, 1, 1, 0.95)
```

6.6 音声、入力、保存

呼び出し	備考
`lt.load_sound(path: str): num`	WAV
`lt.play(sound, volume=1, loop=0): num`	チャンネルを返す。16 個すべて使用中なら −1
`lt.stop(channel)`	
`lt.volume(v)`	
`lt.key(name: str): bool`	押している間。キーボードとゲームパッドを統合
`lt.pressed(name: str): bool`	このフレームで押された
`lt.gamepad(): bool`	
`lt.rumble(low, high, ms)`	
`lt.save(name: str, data: str): bool`	バイナリデータを安全に保存
`lt.load_save(name: str): str?`	**オプショナル値**。読めなければ `nil`

`lt.key` はキーを押している間、真です。`lt.pressed` はキーを押したフレームだけ真です。連続的な移動と一回だけの動作の違いに対応します。認識する入力名は次のとおりです。

```
left right up down z x c space return escape a s d
w
```

キーボードとゲームパッドはこの名前へ統合されます。パッドの十字キーとスティックは方向に、ボタン A は `z`、B は `x`、Y は `c`、Start は `return` に対応します。`lt.key("z")` を使ったゲームは、追加のコードなしでどちらの入力でも動きます。

この表でオプショナル値を返すのは `lt.load_save` だけです。言語全体の動機を語る章がこの呼び出しを中心にしているのは、偶然ではありません。シグネチャは `load_save(str): str?` です。`nil` を扱うまで、コンパイラは結果を使うことを許しません。それこそが第 4 章の主題です。

6.7 決定性

wick と lantern は、同じプログラムに同じ入力を与えると、どの機種でも同じフレームを生成するように作っています。二つの仕組みがそれを実現し、その上にテストの習慣を築いています。

決定的な乱数。 組み込みの `rand()` は、固定の xorshift64* 生成器から `[0, 1)` の一様な値を返します。*どの機種でも同じ列*です。時刻をシードにするモードはありません。実行ごとに変化をつけたい場合は、自分で選び記録できる数値を `srand(seed)` に渡します。

```
srand(7)                    // same layout every run, on every
    machine
let stars: list<num> = []
for i in 0..40 {
  push(stars, rand() * 400)
  push(stars, rand() * 240)
}
```

列が固定なので、シードを指定した実行は同じように再現されます。スクリーンショットのテストが意味を持つ理由です。「ランダムな」星空も、CI と手元で同じ星空になります。

固定刻みの時刻。 `lt.time()` は起動後の秒数を返します。環境変数 `LANTERN_FIXED_DT` があると、フレーム時刻は実際の時計の進み方によらず、固定の刻みで進みます。決定的な `rand()` と組み合わせると、ゲームプレイ全体をフレームごとに再現できます。

スクリーンショットの保存。 `LANTERN_SHOT=/tmp/x` を指定すると、エンジンは 60 フレーム目を BMP として保存して終了します。`LANTERN_FIXED_DT=1` も指定すれば、どこでもバイト単位で同じ画像になります。この二つが「まだ正しく見えるか」を機械的な検査に変えます。

```
LANTERN_SHOT=/tmp/x LANTERN_FIXED_DT=1 ./build/lantern
    games/mygame
```

6.8 検証を日常にする

決定性が役に立つには、ゲームが自分を動かし、確認できる必要があります。wick はそのために二つの組み込み関数を用意しています。

env による自動プレイ。 `env(name): str?` は環境変数を読みます。未設定なら `nil` のオプショナル値です。ゲームが自動プレイのスイッチを

公開し、それで分岐すれば、CI は固定シードでゲームを自動操作できます。

```
let AUTO = env("SHOWCASE_AUTO") != nil   // deterministic
    self-play (CI)
if AUTO { srand(7) }
```

Lantern Night の `AUTO` はキーボードを読む代わりに、最も近い消えたランプへプレイヤーを動かします。そのため、画面を表示しない CI でも実際の決定的なラウンドをプレイし、60 フレーム目のスクリーンショットを本当の回帰テストにできます。

check による表明。 `check(cond: bool, msg: str)` は実行時の表明です。`cond` が偽ならフレームを止め、エラー画面に `check failed: <msg>` を表示します。wick のテストスイートは、この仕組みで構成しています。既知の状態を用意するシーンで結果を `check` し、固定刻みの時刻で動かすことで、正確な結果を得ます。

```
check(len(lamp_x) == 4, "expected four lamps")
check(nearest_unlit() == -1, "all lamps should start lit")
```

`env` で制御する自動プレイ、決定的な `rand`、固定刻みの時刻、`check` を組み合わせると、どの機種でも同じ方法で自分をテストするゲームになります。初めから言語とエンジンを決定性のために設計することの、実用的な成果です。

第 7 章

内部の仕組み

本章では実装を開いて見ます。wick のゲームを書くために必要な知識ではありませんが、言語の設計は、その仕組みを通じて最もよく理解できます。wick の仕組みは、一章で率直に説明できるほど小さいものです。字句解析器、コンパイラ、仮想マシン、ガベージコレクタ、組み込み関数を含む言語全体が、次の三つのファイルにある、依存ライブラリのない約二千行の C++ です。

`wick/wick.hpp`	組み込み用 API（約 120 行）
`wick/wick_front.cpp`	字句解析器と一回走査の型付きコンパイラ → バイトコード
`wick/wick_vm.cpp`	スタック VM、マーク・スイープ GC、組み込み関数、シグネチャ解析器

7.1 一回走査の型付きコンパイル

特徴を決めている選択は、**抽象構文木がない**ことです。コンパイラは *Crafting Interpreters* の *clox* のような一回走査方式です。Pratt 法の式解析器が、進むにつれてバイトコードを生成し、式ごとに静的な `Type` を持ちます。型検査器とコード生成器が木を別々に二回たどるのではなく、同じ走査です。解析器が式を処理し終えると、その値を計算するバイトコードの生成と値の型の決定が、一度の走査でともに終わっています。

言語から見える二つの規則が、この設計から直接生まれます。

使用前の宣言。 コンパイラは名前や呼び出しに到達した時点で解決し、後から前方参照を補う走査はありません。そのため、関数は呼び出すより前に定義し、変数は使うより前に宣言しなければなりません。設計者が制約そのものを目的として選んだのではなく、一回走査のコンパイルの代償です。その代償は小さいものです。

生成時に型が分かること。 式のコードを生成するとき、その型が分かっているので、コンパイラはバイトコードを*特化*できます。性能の説明の中心なので、具体的に見ていきましょう。

7.2 特化した命令

動的な型の VM では、`a + b` は汎用の「加算」命令一つになります。実行時にオペランドのタグを見て、数値の加算、文字列の連結、エラーのどれを行うか決めます。そのタグ検査は、命令を実行するたびに行います。wick はすでに型を知っているため、コンパイル時に*一度だけ*そのコストを払います。

- 二つの `num` への `+` は `OP_ADD`、二つの `str` への `+` は `OP_CONCAT` を生成します。選択はコード生成時に済んでいるので、実行時にタグを調べて選ぶことはありません。

- 大小比較は、オペランドが `num` と分かっているため、数値比較命令 `OP_LT`、`OP_LE`、`OP_GT`、`OP_GE` にコンパイルします。

- ローカル変数はフレームスロット、つまり現在の呼び出しのスタックフレーム内への小さな整数オフセットに解決します。`OP_LOADL` と `OP_STOREL` はスロット番号を受け取り、実行時の名前検索はありません。グローバル変数も `OP_LOADG` と `OP_STOREG` でスロット添字に解決します。

- エンジンのネイティブ関数はコンパイル時に整数 ID に解決します。VM のネイティブ呼び出し `OP_NCALL` は、名前でハッシュテーブルを探すのではなく、ネイティブ関数表への配列添字です。

バイトコードには約四十の命令があります。算術と比較に加え、制御フローのジャンプ `OP_JMP`、`bool` を取り出す `OP_JF`、ループで後方へ飛ぶ `OP_JB`、オプショナル値を実装する二つのジャンプ `OP_JNN`（nil でなければジャンプ）と `OP_ISNIL` があります。呼び出し命令は wick 関数用の `OP_CALL` とネイティブ関数用の `OP_NCALL`、コンテナ命令は `OP_LIST`、`OP_MAP`、添字の取得・設定の組 `OP_IGET`・`OP_ISET` と `OP_MGET`・`OP_MSET` です。特化した組み込み関数には `OP_LEN`、`OP_TOSTR`、`OP_TONUM`、`OP_LPUSH`、`OP_LPOP` があります。いずれも、周囲のコードについて型検査器が証明した事実に対応します。

7.3 スタック VM

実行時の仕組みは、伝統的なスタックマシンです。定数プール、値のスタック、呼び出しフレームのスタックがあります。各フレームは、値スタック内でスロットが始まる基点と、命令ポインタを記録します。フレームを積むときには引数はすでにスタック上にあり、呼び出された関数のローカル変数はそのすぐ上のスロットに置かれます。戻るとスタックをフレームの基点まで縮めるので、トップレベルの文の間では空になり、特に大切なことに、フレームの間でも空になります。

実行時エラーには、コンパイル中に作るバイトコードオフセットと行番号の対応表を使います。範囲外の添字や失敗した `check` などのエラーを VM が出すとき、ソースの行を報告し、エンジンのエラー画面でその行を示せます。wick の実行時エラーが生のバイトコードオフセットではなく `file:line: message` になる理由です。

JIT はありません。これは未完成なのではなく、意図的です。400×240 のゲームロジックにはインタープリタで十分な速さがあります。フレームごとの処理量は少なく、特化した命令が単純なインタープリタの余分なコストを取り除きます。さらに、エンジンは iOS をはじめ、実行時のコード生成と実行を禁止するプラットフォームを対象にしています。JIT があると、wick はそこで許可されません。特化したバイトコードのインタープリタは、仕組み上どこでも使え、ウォームアップや最適化解除による急な性能低下がなく、性能を予測できます。

7.4 フレーム境界でのガベージコレクション

wick は文字列、リスト、マップという三種類のヒープオブジェクトを、マーク・スイープ方式のコレクタで管理します。汎用のものと違い、ゲーム向けにしているのは、いつ動くかです。自発的には決して動かず、ホストが `wick::collect()` を呼ぶときだけ動きます。lantern は画像を表示した後、毎フレーム正確に一度呼びます。

そのため、回収が描画の途中に割り込むことはありません。フレームが画面に出てから次が始まるまでという、停止が見えないタイミングだけで回収します。回収の最悪時のコストは、一フレームが作ったゴミの量で抑えられます。状態を再利用するゲームループでは、その量は小さいものです。Lua のゲームでフレームを落とすような、世代の状態に応じて予測できないタイミングで起きる停止はありません。

コレクタのルートは単純で、それは言語が小さいからこそです。マーク処理は定数プール、グローバル変数、VM の値スタックから始まり、

リストとマップ内の参照を追います。v0.3 にはクロージャもアップバリューもないため、捕捉した環境をたどる必要がなく、オブジェクトグラフは浅い構造です。また、値スタックはフレーム間で空になるため、回収時に生きているルートはグローバル変数と定数、つまり長く使う状態だけで、マーク処理の仕事は少なくなります。

7.5 wick の組み込み

wick は組み込むために作られています。公開 API は `wick.hpp` だけで、組み込み側が内部ヘッダに触れる必要はありません。ライフサイクルは、作成、環境の登録、プログラムの読み込み、毎フレームの関数呼び出し、安全な時点での回収、ホットリロードのためのリセットです。

```
#include "wick.hpp"

wick::VM* vm = wick::create();
std::string err;

// typed natives: a signature DSL the compiler enforces
wick::addNative(vm, "game", "spawn(num, num, str): num",
    [](wick::VM& vm, const wick::Value* a, int) {
        int id = mySpawn(a[0].d, a[1].d, wick::getStr(a[2]));
        return wick::Value::num(id);
    }, err);
wick::addConst(vm, "game", "MAX", 64);

wick::load(vm, source, "main.wick", err);   // compile + run top
    level
wick::call(vm, "update", dt, true, err);    // each frame
wick::collect(vm);                          // at YOUR safe point
wick::reset(vm);                            // hot-reload support
```

シグネチャ DSL。 ネイティブ関数にはシグネチャ文字列を登録します。コンパイラは一度解析し、すべての呼び出し箇所で強制します。文法は次のとおりです。

```
name(type[, type][, type = default]) [: type]
```

型は `num`、`bool`、`str`、`list` で、末尾の `?` はオプショナル値を表します。この一つの文字列によって、`lt.load_save(str): str?` は保存の読み込

みがオプショナル値を返すとコンパイラに教えます。エンジン呼び出しに対する第 4 章のオプショナル値の規律全体を、シグネチャ内の二文字が支えています。既定値は数値リテラルでなければならず、末尾の引数を呼び出し箇所で省略可能にします。コンパイラが補うため、ネイティブ関数は常にすべての引数を受け取り、`argc` を調べる必要がありません。名前空間 `"lt"` や `"game"` の下に登録すると、関数名が修飾されます。名前空間に null を渡すとグローバルスコープに置きます。`floor` や `sqrt` のような組み込み関数の登録方法です。

ネイティブ関数からのエラー報告。
ネイティブ関数は `wick::setError(vm, msg)` を呼んで失敗を知らせます。VM は呼び出し箇所を示す `file:line` の実行時エラーに変換するので、他の実行時エラーと同じ報告になり、同じエラー画面に出ます。値の補助関数 `makeStr`、`getStr`、`listLen`、`listGet` は、ネイティブ関数が受け取り返すヒープ上の値の読み取りと構築を担います。

ホットリロード。 `wick::reset(vm)` は、グローバル変数、関数、コンパイル済みプロトタイプというすべてのプログラム状態を消去し、登録済みの環境であるネイティブ関数と定数は残します。プログラムを解体し、バインディングを維持し、新しいソースを読み込む。ホットリロードに必要なことそのものです。`reset` は新しいプログラムを読み込む前に古いリソースを解放するので、再読み込みはリークしません。チュートリアルで約束した性質です。

どれも大きな仕組みではありません。組み込み API は約 120 行で、実装全体は約二千行です。小ささは本書を通じて繰り返す主題ですが、ここで最もはっきり見えます。静的な型を持ち、オプショナル値を検査し、決定的に動くスクリプト言語に、ガベージコレクタと型付きの外部関数インターフェースが備わっています。それでも午後だけで読み切れ、全体を自分で把握できます。

第 8 章

設計とトレードオフ

言語は判断の集合であり、判断には利益だけでなく代償もあります。本章では wick の判断を率直に論じます。同じ実際のゲームの wick 版と Lua 版を並べ、今でも Lua の方が読みやすい点を明言し、v0.3 の省略が恥ずべき欠落ではなく意図的な選択であることを支える、ロードマップの考え方を説明します。

8.1 同じゲームを二つの言語で

Lantern Night がその証拠です。Lua の `games/showcase` と wick の `games/showcase_wick` という、意図的に別々にした二つのプロジェクトとして、両言語で提供しています。同じ実際のゲームで比較し続けられるよう、並べて維持しています。二つをロゼッタストーンのように読み比べてください。三つの抜粋で議論を示します。

8.1.1 最高得点の読み込み

第 4 章のバグが、本来の場所にあります。Lua 版は、最初のフレームでクラッシュする状態で出荷されました。

```
-- Lua: a 0-byte save makes load_save return "" (truthy!), the
-- fallback never fires, and tonumber("") = nil poisons
    everything.
local best = tonumber(lt.load_save("lanternnight_best") or "0")
```

wick 版ではそのバグを表現できません。ファイルがない場合と空の場合という、値がない二つの可能性をどちらも処理する必要があるため、下の行だけがコンパイルできます。

```
// wick: the ONLY version the compiler accepts.
let best = num(lt.load_save("lanternnight_wick_best") ?? "") ?? 0
```

ここでは wick が明確に優れています。安全なコードはバグのある Lua のコードより短く、安全でないコードはコンパイルできません。

8.1.2 必要性が示されたレコード

Wick 0.1 は、ランプや小道具をスカラーの並行リストで表していました。コンパイラを小さく保てる一方、一つの論理オブジェクトが複数のリストに散らばります。KORA のキッチンで、その代償が具体的になりました。五つのリストを同期させる必要があったのです。そのため Wick 0.2 は平坦なレコードを取り入れました。第 5 章で、検査されるフィールドとリストへの格納を説明します。Lua は今でも、より豊かな入れ子のテーブルとメソッドを扱えます。Wick はその範囲すべてに対応していると主張しません。

Wick 0.3 も同じ規則に従います。プロセッサを組み立てるプロジェクトには、マスク、シフト、レジスタの折り返し、読みやすいバスの値が必要です。Bit Lab はその操作を直接使い、第 9 章はその成果を説明します。新しい関数は既存の型付きネイティブ関数の仕組みを使い、式の優先順位を変更しません。

8.1.3 複数の戻り値

こちらは小さな不便です。Lua は最も近いランプとその距離を一緒に返します。

```
local i, dist2 = nearest_unlit()
```

wick には複数の戻り値がないので、二つの問い合わせに分け、「ランプがない」は第二の値の欠落ではなく、番兵値の添字にします。

```
let i = nearest_unlit()      // returns the index, or -1
let d = dist2_to(i)          // second query, explicit
```

記述は長くなりますが、本質的に悪くなったわけではありません。分割は機械的で、番兵値はよく知られた書き方です。複数の戻り値はロードマップにあります。今ないことは不便であって、危険ではありません。

8.2 書かなくなるもの

そうした代償と引き換えに、動的な言語が要求する防御的なコードは実際に減ります。Lantern Night を wick に移植すると、以下はそのままソ

ースから消えました。

- 保存の読み込みや解析を囲む `if x ~= nil` の連なり。コンパイラが値の欠落の可能性を追跡するので、境界で一度 `??` を書けば、その後は普通の `T` です。

- `math.` の接頭辞。`sin`、`floor`、`rand` などは名前空間のない組み込み関数です。

- 整数を末尾の小数なしで表示するためだけの `string.format`。`str(n)` は整数をすっきり表示します。

- エンジン呼び出しの引数順を過度に心配すること。引数の数と型をコンパイル時に検査し、メッセージに呼び出し名と引数番号が出るので、画面がおかしく見えてからではなく、書いた瞬間に誤りが分かります。

8.3 変わらないもの

言語を変えても影響しなかった点を明確にしておきます。wick を採用するコストの範囲が分かるからです。エンジン API は呼び出しごとに同一で、`lt.draw(...)` は両方で同じ行です。`update(dt)` と `draw()` というフレームの契約も同じです。ホットリロードとエラー画面も同じように動きます。そして二つのゲームは同じフレームを描画します。これは比喩ではなく、第 6 章の決定的なスクリーンショットテストで、wick 版と Lua 版に同じピクセルを要求しています。wick が変えたのは言語で、エンジンや開発の流れではありません。

8.4 v0.3 の制約

wick は意図的に Lua 1.0 のような小ささにしています。以下の省略はすべて、現在の回避策とロードマップ上の位置づけを持つ既知の欠落です。後から発覚する驚きではありません。v0.3 の基準は明確で狭く、実際の lantern ゲームを動かし、Lua の種類のバグをなくし、約二千行を保つことでした。

v0.3 にないもの	現在の回避策
クロージャ、入れ子・第一級の関数	トップレベルの `fn` とモジュールのグローバル変数
複数の戻り値	二つの関数、または戻り値の外の状態
入れ子のコンテナ（`list<list<num>>`）	平坦なリストの添字計算
モジュール・インポート（`main.wick` 一つ）	まだアプリ基盤ではなく、ゲームのスクリプト
`for x in list` の反復	`for i in 0..len(xs)`
文字列の添字・スライス・メソッド	HUD には `len`、`+`、`str()` で対応
戻り値を返す経路の完全な検査	型付き関数が末尾まで進むと `nil`。必ず返すこと
グローバル変数・`and` の連なりの絞り込み	`??`、または先にローカル変数へコピー
指数表記の数値リテラル	10 進表記。16 進・2 進は対応済み

二つの点は別に述べる必要があります。これは制約ではなく選択であり、ロードマップでも覆さない判断だからです。

JIT を使わない。 仕組み上 iOS で許可され、ウォームアップなしに決定的な性能を得られます。これは恒久的な特徴です。

時刻をシードにした乱数を使わない。 再生と CI のスクリーンショットを可能にするため、決定性を既定にします。実行ごとに変化が必要なら、記録できる数値で自分で `srand()` を設定します。これも恒久的です。

8.5 ロードマップの考え方

次を決める規則は、この言語が従う核心なので、原則として明記します。*wick は本当に必要なものだけを持ち、持つこと自体を目的に加えません。*実際のゲームのコードで、それがないことによる痛みが示されたときに初めて機能を取り入れます。対になった Lantern Night の二つの版は、常設の証拠の仕組みです。言語を大きくする前に、実際のゲームか焦点を絞った例で、機能の欠落による痛みが現れる必要があります。オプショナル値が本書にあるのは、出荷したゲームのクラッシュが要求したからです。クロージャがないのは、出荷したプログラムがまだその不

在の代償を払っていないからです。他の言語と機能をそろえることは、追加の理由にはしません。

全体をまとめる考えは「Lua 1.0 のように小さく」です。Lua はメタテーブル、コルーチン、エコシステムから始まったのではありません。小さく、明快で、組み込みやすい核から始まり、実際の使用が各追加を正当化するにつれて成長しました。wick は意図的に同じ道を進みます。表にある省略はすべて、既存の型付きフロントエンドの背後に追加できます。クロージャと複数の戻り値はコンパイラを考え直す必要はなく、拡張で対応できます。どれも実際のゲームが、規模を増やすだけの必要性を示したときに加え、それ以前には加えません。

だからこそ制約を率直に述べます。欠落を隠す言語は、最悪の時点でユーザーが発見するよう招いていることになります。wick は一覧にし、それぞれに回避策を示し、ロードマップに置きます。プログラムの失敗のしかたは本番で発見するのではなく、事前に見えるべきだという思想が、言語全体の主張だからです。言語は、ユーザーのコードに求めるのと同じ率直さを、ユーザーへ示す責任があります。

第 9 章

ビット、バイト、バス

Wick 0.3 は、次の具体的なプロジェクトから生まれました。Intel 8080 を目標に、プレイヤーがプロセッサを組み立てるゲームです。そのゲームの言語には、ビットを目に見える形にし、レジスタ幅を明示する必要があります。本章では小さな部品を組み立てます。CPU 全体やそのタイミングを実装するものではありません。

9.1 意図するビットを書く

```
let opcode = 0x76
let sign_mask = 0b10000000
let last_address = 0xFFFF
check(opcode == 118, "hexadecimal")
check(sign_mask == 128, "binary")
```

16 進と 2 進の接頭辞は、大文字でも小文字でも使えます。数字は基数に適合し、値全体は符号なし 32 ビットに収まる必要があります。数字のない接頭辞、アンダースコア、接頭辞付きの小数、0xFFFFFFFF を超える値はコンパイルエラーです。10 進数の規則は変わりません。これらの値はすべて引き続き num で、整数型は追加していません。

9.2 ゲートとマスク

ビット関数は bit_and、bit_or、bit_xor、bit_not、bit_shl、bit_shr の六つです。最初の三つは二つの値を組み合わせます。NOT は 32 ビットすべてを反転します。左シフトはビット 31 より外のビットを捨て、右シフトは 0 で埋めます。オペランドは 0 から 4294967295 までの整数、シフト数は 0 から 31 までの整数でなければなりません。

```
let wires = 0b10100101
```

```
check(bit_and(wires, 0x0F) == 5, "low nibble")
check(bit_xor(wires, 0x80) == 0x25, "toggle bit 7")
check(bit_shr(wires, 7) == 1, "read high bit")
check(bit_shl(1, 31) == 0x80000000, "high bit")
```

型や引数の数の誤りはコンパイルエラーです。小数のオペランドや 32 のシフト数など、数値の範囲の誤りは、Wick の呼び出し箇所を示す実行時エラーです。VM は C++ の整数変換やシフトを行う前に値を検証します。NaN と無限大も拒否します。

これは普通の型付き関数です。追加しても演算子の優先順位は変わらず、and、or、not は引き続き真偽値に対して動きます。

9.3 レジスタには幅がある

u8(n) は安全に表現できる整数を 256 の剰余で折り返し、u16(n) は 65536 の剰余で折り返します。どちらも負の値を受け付けます。入力は有限な整数で、-9007199254740991 以上 9007199254740991 以下でなければなりません。普通の算術は倍精度のままです。折り返しは、指定した場所でだけ行います。

```
record Register { value: num, name: str }
let a = Register { value: 0xFF, name: "A" }
let wide = a.value + 1
let carry = wide > 0xFF
a.value = u8(wide)
check(a.value == 0 and carry, "keep the carry")
check(u8(-1) == 255, "byte underflow")
check(u16(0xFFFF + 1) == 0, "PC rollover")
check(u8(bit_not(0x0F)) == 0xF0, "8-bit NOT")
```

フラグを計算し終えるまでは、幅の広い結果を保持します。先に折り返すと、プレイヤーが見る必要のあるキャリーそのものを消してしまいます。8 ビットの ADD では、ゼロは result == 0、符号はマスク 0x80 で検査し、補助キャリーは下位 4 ビットの和が 0x0F を超えるかで検査します。パリティは折り返した結果の立っているビットを数え、偶数なら真です。

9.4 ペアとメモリ

上位バイトと下位バイトを合わせると、16 ビットのアドレスになります。

```
let h = 0x80
let l = 0x05
let address = bit_or(bit_shl(h, 8), l)
check(address == 0x8005, "register pair")
check(bit_shr(address, 8) == h, "high byte")
check(u8(address) == l, "low byte")
let memory: list<num> = []
for i in 0..65536 { push(memory, 0) }
memory[address] = 0x76
check(memory[0x8005] == 118, "memory write")
```

このリストは新しいコンテナ型を加えずに、アドレス空間をモデル化します。要素は今でも数値なので、バイトを書き込む境界で `u8(value)` を使います。詰めて格納するバイトバッファではなく、ハードウェアのクロック速度も保証しません。ゲームは描画フレームとは独立して、意図したシミュレーションのステップでプロセッサを進められます。

9.5 状態を読みやすくする

`hex(value, width=1)` は接頭辞なしの大文字の桁を返し、`bin(value, width=1)` は 2 進の桁を返します。値の範囲は符号なし 32 ビットです。幅は最小桁数を指定する整数で、16 進では 1–8、2 進では 1–32 です。表示欄に合わせて値を切り捨てることはありません。

```
check(hex(10, 2) == "0A", "byte label")
check(bin(5, 8) == "00000101", "bus label")
check(hex(256, 2) == "100", "overflow stays visible")
```

9.6 作業台と次のゲーム

`./build/lantern games/bitlab` を実行すると、小さな Wick の作業台でこれらの操作を確認できます。矢印キーで入力レジスタを選択・編集し、Z で ADD、AND、XOR を切り替えます。入力ビットをタッチまた

はクリックすると反転します。フラグは 8080 の ADD、ANA、XRA の動作に従い、ANA の補助キャリー規則も含みます。どちらかの入力のビット 3 を反映し、常にフラグを立てる 8085 の規則とは異なります。参照資料は Intel の *8080/8085 Assembly Language Programming* マニュアル（1977 年）、第 1 章「Auxiliary Carry Flag」です。

Bit Lab は言語リリースのための例です。プレイヤー向けの回路エディタ、進行システム、命令デコーダ、完全なプロセッサの検証は、引き続きゲーム側の作業です。ホストに回路シミュレータを隠しているのではありません。例のロジックは Wick のコードです。

アップグレード。 新しい十個の組み込み関数名と同じ名前のユーザー関数は、コンパイル時に拒否します。衝突するユーザー関数は改名してください。このリリースでは、インポート、クロージャ、入れ子のレコード、文字列の添字、新しい数値型は導入していません。

付録 A

文法

以下の EBNF は、構文リファレンスとリファレンスコンパイラのフロントエンドをもとにした wick v0.3 の文法です。終端記号は引用符で囲み、`{ }` は 0 回以上の繰り返し、`[ ]` は省略可能、`|` は選択肢の区切りを表します。コメントと空白は字句解析器が扱うため、ここには示していません。式の規則の優先順位は、最も弱い `orExpr` から最も強い `postfix` まで、第 3 章の表と一致します。

```
program        = { toplevel } ;
toplevel       = recordDecl | fnDecl | statement ;
recordDecl     = "record" ident "{" field { "," field } "}" ;
field          = ident ":" scalar ;

fnDecl         = "fn" ident "(" [ params ] ")" [ ":" type ]
    block ;
params         = param { "," param } ;
param          = ident ":" type ;
block          = "{" { statement } "}" ;

statement      = letStmt | assign | callStmt | ifStmt | whileStmt
               | forStmt | "break" | "continue" | returnStmt ;
letStmt        = "let" ident [ ":" type ] "=" expr ;
assign         = lvalue "=" expr ;
lvalue         = ident [ "[" expr "]" ] [ "." ident ] ;
callStmt       = call ;
ifStmt         = "if" expr block { "elif" expr block } [ "else"
    block ] ;
whileStmt      = "while" expr block ;
forStmt        = "for" ident "in" expr ".." expr block ;
returnStmt     = "return" [ expr ] ;

type           = element [ "?" ]
               | "list" "<" element ">" [ "?" ]
               | "map"  "<" element ">" [ "?" ] ;
```

```
scalar         = "num" | "bool" | "str" ;
element        = scalar | ident ;  (* ident: declared record
    type *)

expr           = orExpr ;
orExpr         = andExpr    { "or"  andExpr } ;
andExpr        = coalesce   { "and" coalesce } ;
coalesce       = equality   { "??"  equality } ;
equality       = comparison { ( "==" | "!=" ) comparison } ;
comparison     = addition   { ( "<" | "<=" | ">" | ">=" )
    addition } ;
addition       = term       { ( "+" | "-" ) term } ;
term           = unary      { ( "*" | "/" | "%" ) unary } ;
unary          = ( "-" | "not" ) unary | postfix ;
postfix        = primary { "(" [ args ] ")" | "[" expr "]" | "."
    ident } ;
primary        = number | string | "true" | "false" | "nil"
               | qualified | ident | "(" expr ")"
               | listLit | mapLit | recordLit ;
recordLit      = ident "{" ident ":" expr { "," ident ":" expr }
    "}" ;
qualified      = ident "." ident ;         (* e.g. lt.draw *)
call           = ( ident | qualified ) "(" [ args ] ")" ;
args           = expr { "," expr } ;

listLit        = "[" [ expr { "," expr } ] "]" ;
mapLit         = "[" string ":" expr { "," string ":" expr } "]"
    ;

ident          = letter { letter | digit } ;    (* letter
    includes _ *)
number         = digit { digit } [ "." digit { digit } ]
               | ( "0x" | "0X" ) hexDigit { hexDigit }
               | ( "0b" | "0B" ) bit { bit } ;
hexDigit       = digit | "a".."f" | "A".."F" ;
bit            = "0" | "1" ;
string         = '"' { char | escape } '"' ;
escape         = "\n" | "\t" | '\"' | "\\" ;
```

文法についての補足です。空の `listLit`（`[]`）は構文上は合法ですが、要素型を推論できないため、それを含む `let` に型注釈が必要です。`mapLit` には文字列リテラルのキーが必要です。`forStmt` の `..` は一つのトーク

ンで、字句解析器は `1..5` が `1`、`..`、`5` になることを保証します。型名は `type` の位置でだけ認識する文脈依存の名前であり、予約語ではありません。

付録 B

コンパイルエラー一覧

wick の診断はすべて `file:line: message` の形で、エンジン内のエラー画面に現れます。ファイルを直して保存すると、ゲームがホットリロードされます。以下の表では、原因ごとにコンパイル時のメッセージを挙げ、意味と一般的な対処を示します。リファレンスコンパイラの表現には細かな違いがあるかもしれませんが、ここに示す形を基本とします。

メッセージの形	意味と修正方法
`expected str, got str?`（型は例）	普通の値が必要な場所に、オプショナル値または異なる型を使った。`??` で中の値を取り出すか、`if x != nil { }` で絞り込む。
`if condition must be bool (wick has no truthiness)`	数値や文字列に `if x` を使った。`if x > 0`、`if s != ""` のように比較を書く。
`while condition must be bool`	上記と同じ、`while` の条件の誤り。
`unknown variable 'x' --- wick has no implicit globals; use let`	未宣言の名前に代入した。多くはタイプミス。`let` を加えるか、つづりを直す。
`unknown variable 'x' (declare it with let)`	未宣言の名前を読んだ。宣言するか、つづりを直す。
`'+' needs num+num or str+str (use str(x))`	異なる型を `+` で結んだ。`"n=" + str(n)` のように明示的に変換する。
`'-' needs num operands / arithmetic needs num operands`	算術演算子に `num` 以外のオペランドを渡した。
`f() argument 3: expected num, got str`	エンジンまたは自作関数の型付き呼び出しで型が不一致。シグネチャを確認する。

次のページに続く

メッセージの形	意味と修正方法
`f() needs at least N arguments / takes N arguments`	引数の数が不一致。シグネチャを確認する。
`unknown function 'f' (wick requires declare-before-use)`	定義より上で呼び出した。`fn` を上に移す。
`can't infer a type from nil; annotate: let x: str? = nil`	単独の `nil` で初期化した。型注釈を加える。
`empty [] needs a type: let xs: list<num> = []`	型のない空リテラル。`let` に型注釈を加える。
`list elements must share one type`	`[1, "a"]` のように型が混在するリテラル。リストは同じ型の要素だけを持つ。
`list elements must be num, bool, or str`	許可されない型のコンテナ要素。
`comparing non-optional to nil`	普通の型に `x != nil` を使った。`nil` にはならないので、検査を削除する。
`'??' left side must be an optional`	`nil` になり得ない `a` に `a ?? b` を使った。`??` を削除する。
`'??' default must be T`	既定値の型がオプショナル値の元の型と一致しない。
`'x' already declared in this scope / 'x' already declared`	`let` が重複。代入するか、名前を変える。
`parenthesize this condition: (a != b) and ...`	絞り込みのパターンと `and`/`or` が混在。括弧を加える。
`break outside a loop / continue outside a loop`	囲むループのないループ制御文。
`fn declarations can't nest`	別の `fn` の中の `fn`。v0.3 ではトップレベルのみ。
`return outside a function`	トップレベルの `return`。
`void function can't return a value`	戻り値の型を宣言していない関数内の `return expr`。

次のページに続く

メッセージの形	意味と修正方法
`this function must return T`	値を返す関数内の、単独の `return` または型の不一致。
`map keys must be str literals`	マップリテラルに計算したキーを使った。`m[k] = v` で構築する。
`container elements must be num, bool, or str`	入れ子のコンテナ。平坦化する（第 5 章）。
`only lists and maps can be indexed`	リストでもマップでもない値に添字を使った。
`'and'/'or' needs bool operands`	論理演算子に `bool` 以外のオペランドを使った。
`unexpected character 'x'`	字句解析器が認識しないバイト。

実行時エラー

静的な型によって上記の種類の誤りをコンパイル時に取り除けますが、実行時にしか分からない値に依存する次の誤りは取り除けません。いずれもフレームを止め、行番号付きでエラー画面に出ます。

メッセージ	原因
`list index N out of range (len M)`	範囲外の `xs[i]` の読み取りまたは書き込み。
`check failed: <msg>`	自作の `check()` による表明が失敗。
`load_texture/mesh/sound failed: <path>`	存在しない、または形式の不正なアセット。

付録 C

早見表

組み込み関数

名前空間なしで、常に使えます。汎用の組み込み関数 `len`、`str`、`num`、`push`、`pop` は、コンパイル時に静的な型に応じて処理を選びます。

シグネチャ	備考
`len(x): num`	`str`、`list`、`map` の長さ
`str(x): str`	`num`、`bool`、`str` を文字列へ。整数は小数なし
`num(s: str): num?`	数値として解析。失敗時は `nil`（`""` を含む）
`push(xs: list<T>, v: T)`	末尾に追加
`pop(xs: list<T>): T?`	最後を取り除いて返す。空なら `nil`
`floor(x: num): num`　`ceil(x: num): num`	
`abs(x: num): num`　`sqrt(x: num): num`	
`min(a: num, b: num): num`	
`max(a, b): num`	
`sin(x: num): num`　`cos(x: num): num`	ラジアン
`atan(y: num, x: num): num`	二引数の逆正接
`pi`	定数、3.14159…
`rand(): num`	`[0, 1)` の一様な値。どこでも同じ列
`srand(seed: num)`	再現可能な実行のためシードを再設定
`env(name: str): str?`	環境変数を読む。未設定なら `nil`
`check(cond: bool, msg: str)`	実行時の表明

lt.* エンジンインターフェース

呼び出し	備考
フレームと画面	
`lt.W  lt.H`	定数 400、240
`lt.clear(r,g,b)`	色と深度を消去
`lt.time(): num`	起動後の秒数
`lt.screenshot(path: str)`	フレームを BMP として保存
`lt.quit()`	
`lt.escape_quits(enable: bool)`	
3D シーン	
`lt.camera(ex,ey,ez, tx,ty,tz, fov=55)`	位置と注視点
`lt.light(dx,dy,dz, ambient=0.35)`	平行光源
`lt.point_light(i, x,y,z, radius, r=1,g=1,b=1)`	スロット 0–3
`lt.fog(start, end, r,g,b)`	深度に線形
メッシュ	
`lt.cube()  lt.plane(segs=1)`	単位プリミティブ
`lt.sphere(seg=12)`	
`lt.cylinder(seg=16)`	
`lt.cone(seg=16)`	
`lt.mesh(verts: list<num>): num`	頂点ごとに浮動小数点数 12 個
`lt.load_mesh(path: str): num`	Wavefront OBJ
`lt.draw(m, x,y,z, rx,ry,rz, sx,sy,sz, r,g,b, tex=-1)`	照明を付けて描画
`lt.draw_lerp(a, b, t, …, tex=-1)`	キーフレーム補間
`lt.billboard(tex, x,y,z, w,h, …)`	カメラを向く四角形
`lt.shadow(x,y,z, radius, alpha=0.35)`	接地感を出す簡易の影
2D	
`lt.rect(x,y,w,h, r,g,b, a=1)`	塗りつぶし、アルファ合成
`lt.load_texture(path: str): num`	BMP。マゼンタは透明
`lt.sprite(tex, x,y, sx=1,sy=1)`	左上を基準点に配置

呼び出し	備考
`lt.sprite_ex(tex, cx,cy, sx,sy, rot,r,g,b,a)`	中心基準、回転、着色
`lt.sprite_uv(tex, x,y,w,h, u0,v0,u1,v1)`	アトラス内の部分矩形
`lt.print(text: str, x,y, r=1,g=1,b=1,a=1)`	8×8 フォント
音声、入力、保存	
`lt.load_sound(path: str): num`	WAV
`lt.play(sound, volume=1, loop=0): num`	チャンネル、または −1
`lt.stop(channel)` `lt.volume(v)`	
`lt.key(name: str): bool`	押している間
`lt.pressed(name: str): bool`	このフレームで押された
`lt.gamepad(): bool`	
`lt.rumble(low, high, ms)`	
`lt.save(name: str, data: str): bool`	バイナリデータを安全に保存
`lt.load_save(name: str): str?`	オプショナル値。読めなければ `nil`

入力名：`left right up down z x c space return escape a s d w`。ゲームパッドでは十字キーとスティックが方向に、A、B、Y、Start が `z`、`x`、`c`、`return` に対応します。

C.1 ビットとレジスタ幅（0.3）

`bit_and`、`bit_or`、`bit_xor`、`bit_not`、`bit_shl`、`bit_shr` は符号なし 32 ビット整数を受け取ります。シフト数は 0–31 の整数です。`u8` と `u16` は安全に表現できる整数を 256 と 65536 の剰余で折り返します。`hex(n, width=1)` と `bin(n, width=1)` は、符号なしの値を最小桁数の 0 埋め付きで表示します。幅はそれぞれ 1–8 と 1–32 で、切り捨てはしません。すべての関数は、実行時に不正な数値範囲を拒否します。第 9 章を参照してください。